TPCT'S TERNA ENGINEERING COLLEGE

An Autonomous Institute Affiliated to University of Mumbai

A Final Year Sem VII Major Project I Report on

"SecureCode AI: A Comprehensive Hands-On Secure Coding Practice and Multi-Engine Evaluation Platform"

Submitted in partial fulfillment of the requirement of
University of Mumbai for the Degree of

Bachelor of Engineering (Computer Engineering)

By

Harsh Jain	**TU3F2324122**
Atharva Kadam	**TU3F2324118**
Sayli Vekhande	**TU3F2324216**
Arya Ratnakar	**TU3F2324040**

Under the Guidance of

Prof. Mohini Misale

Department of Computer Engineering

TERNA ENGINEERING COLLEGE

Nerul (W), Navi Mumbai 400706

(University of Mumbai)

(Academic Year 2026-2027)

TERNA ENGINEERING COLLEGE, NERUL, NAVI MUMBAI

Department of Computer Engineering

Academic Year 2026-27

CERTIFICATE

This is to certify that the Major Project I entitled "SecureCode AI: A Comprehensive Hands-On Secure Coding Practice and Multi-Engine Evaluation Platform" is a bonafide work of:

1) Harsh Jain (ID No: TU3F2324122 / Roll No: A-58)
2) Atharva Kadam (ID No: TU3F2324118 / Roll No: A-54)
3) Sayli Vekhande (ID No: TU3F2324216 / Roll No: A-63)
4) Arya Ratnakar (ID No: TU3F2324040 / Roll No: A-47)

submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the Degree of Bachelor of Engineering (Computer Engineering).

Prof. Mohini Misale	Dr. Kishor Sakure	Dr. Deven Shah
Guide	Head of Department	Principal

Project Report Approval

This Major Project I Report – entitled "SecureCode AI: A Comprehensive Hands-On Secure Coding Practice and Multi-Engine Evaluation Platform" by the following students is approved for the degree of B.E. in "Computer Engineering".

Submitted by:

1) Harsh Jain (ID No: TU3F2324122 / Roll No: A-58)
2) Atharva Kadam (ID No: TU3F2324118 / Roll No: A-54)
3) Sayli Vekhande (ID No: TU3F2324216 / Roll No: A-63)
4) Arya Ratnaker (ID No: TU3F2324040 / Roll No: A-47)

Examiners Name & Signature:

1. _____

2. _____

Date: _____

Place: _____

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

1) Harsh Jain (ID No: TU3F2324122 / Roll No: A-58) _____

2) Atharva Kadam (ID No: TU3F2324118 / Roll No: A-54) _____

3) Sayli Vekhande (ID No: TU3F2324216 / Roll No: A-63) _____

4) Arya Ratnaker (ID No: TU3F2324040 / Roll No: A-47) _____

Date: _____

Place: _____

Acknowledgement

We would like to express our sincere gratitude towards our guide Prof. Mohini Misale and our Major Project Coordinator Dr. Siddharth Hariharan for their invaluable guidance, encouragement, and constant constructive feedback throughout the inception, design, and implementation of SecureCode AI. This project would not have been possible without their continuous support, scholarly direction, and motivation. We thank them for making our engineering endeavor thoroughly engaging and intellectually enriching.

We are deeply thankful to Dr. Kishor Sakure (H.O.D., Department of Computer Engineering) and the faculty members of the Computer Engineering Department for providing access to computing facilities, software resources, and technical consultation. Their scientific rigor and pedagogical standards inspired us to strive for academic excellence and robust software engineering practices.

We take the privilege to express our sincere thanks to Dr. Deven Shah, our Principal, for providing an environment that fosters technological innovation and research. Finally, we thank our peers and families for their steadfast encouragement throughout this endeavor.

1) Harsh Jain (ID No: TU3F2324122 / Roll No: A-58) _____

2) Atharva Kadam (ID No: TU3F2324118 / Roll No: A-54) _____

3) Sayli Vekhande (ID No: TU3F2324216 / Roll No: A-63) _____

4) Arya Ratnaker (ID No: TU3F2324040 / Roll No: A-47) _____

TABLE OF CONTENTS

Sr. No.	Title	Page No.
01	Abstract	08
02	List of Figures	09
03	List of Tables	10
Chapter 1	Introduction	11-14
	1.1 Motivation	11
	1.1.1 Need of the Problem	12
	1.2 Scope of the Project	13
	1.3 Aim and Research Objectives	14
Chapter 2	Problem Statement & Requirements Engineering	15-19
	2.1 Problem Statement	15
	2.2 Features of the System	16
	2.3 Objectives	17
	2.4 System Specifications & Requirements	17
	2.4.1 Functional Requirements	17
	2.4.2 Non-Functional Requirements	18
Chapter 3	Literature Review	20-27
	3.1 Existing Systems	20
	3.2 Limitations of Existing Systems	22
	3.3 Proposed Solution	23
	3.4 Research Gap & Comparative Analysis	24
	3.5 Literature Survey (27 Peer-Reviewed Works)	25
Chapter 4	Design and Implementation	28-44
	4.1 System Architecture	28
	4.2 Working of the System	30
	4.3 Use Case Modeling	31
	4.4 Data Flow Diagrams (Level 0, 1, and 2)	32
	4.5 Evaluation Workflow & Activity Modeling	35
	4.6 Multi-Engine Security Architecture & Consensus	36

	4.7 Algorithms and Mathematical Formulations	37
	4.7.1 Dual-Stage Evaluation Gate Algorithm	37
	4.7.2 Static Analysis Normalization Algorithm	38
	4.7.3 Four-Pillar Overall Learning Score Formulation	38
	4.7.4 Attempt Windowing & Prerequisite Progression	39
	4.8 Relational Data Model & Database Schema	40
	4.9 Class Diagram & Domain Layer Services	41
	4.10 UI Wireframe Design & Interaction Principles	41
	4.11 Software Process Model & Project Timeline	42
	4.12 Risk Mitigation, Monitoring and Management (RMMM) Plan	43
	4.13 Hardware Requirements	44
	4.14 Software Requirements & Technology Stack	44
Chapter 5	Results and Discussions	45-56
	5.1 Experimental Evaluation and Benchmarks	45
	5.2 Landing & Home Page	47
	5.3 Challenge Catalog & Vulnerability Selection	48
	5.4 Learning Hub & Pedagogical Fundamentals	49
	5.5 Interactive Code Editor & Dual-Stage Evaluation	50
	5.6 Attempt History & Remediation Progression	51
	5.7 Submission Analysis & Multi-Layer Feedback	52
	5.8 Analytics Dashboard & Security Metrics	53
	5.9 Vulnerability Trend Analysis & Injection Prioritization	54
	5.10 Industry Platform Comparison & Gap Resolution	55
Chapter 6	Conclusion and Future Scope	57-58
	6.1 Conclusion	57
	6.2 Future Scope	58
04	References (Consolidated Bibliography)	59-62
05	Appendix A: Active Injection Catalog & Rubric	63-65

Abstract

Software security vulnerabilities remain a prevalent and costly threat to global digital infrastructure. While traditional competitive programming and educational coding platforms (such as LeetCode, HackerRank, and Codewars) effectively cultivate algorithmic proficiency, they focus exclusively on functional correctness while ignoring software security vulnerabilities. Conversely, offensive cybersecurity platforms (such as PortSwigger Web Security Academy, TryHackMe, and Hack The Box) emphasize penetration testing and exploitation rather than the constructive software engineering disciplines required to write secure source code. Furthermore, modern generative AI coding assistants frequently produce functionally acceptable yet insecure code, exacerbating the vulnerability epidemic.

This report presents SecureCode AI, an interactive, web-based secure coding practice and automated assessment platform engineered to cultivate defensive programming competencies from the ground up. SecureCode AI implements a novel dual-stage evaluation architecture: Stage 1 executes dynamic functional unit testing within an isolated sandbox (via Judge0), while Stage 2 performs multi-engine static and AI-assisted security analysis combining abstract syntax tree (AST) rules (Semgrep), bytecode dataflow inspection (SpotBugs/FindSecBugs), and contextual large language model advisory (Groq LLaMA-3). To prevent trial-and-error guessing and superficial code modification, SecureCode AI introduces an active catalog of 45 staged injection challenges (spanning SQL, NoSQL, OS Command, LDAP, XPath, Header/CRLF, and Code Injection), a rate-limiting five-attempt window, interactive conceptual understanding quizzes, and a multi-factor learning scoring engine that holistically weights Functional Correctness (25%), Security Correctness (45%), Conceptual Understanding (15%), and Independent Problem Solving (15%).

The system was engineered using Spring Boot 3, Java 25, PostgreSQL, Flyway migrations, and a responsive web application client. Empirical validation across 80 automated unit, integration, and security test suites demonstrates zero false passes on vulnerable code, flawless consensus arbitration across disparate static analysis outputs, and sub-500ms median evaluation latency for local static engines. SecureCode AI bridges the critical pedagogical chasm between algorithmic programming and software security, empowering developers to construct verifiably secure systems before deployment.

Keywords: Secure Code Generation, Automated Programming Assessment Systems (APAS), Static Application Security Testing (SAST), Injection Attacks, Dual-Stage Evaluation, Learning Analytics, Large Language Models (LLM), Software Engineering Education.

List of Figures

Figure No.	Name of Figure	Page No.
Chapter 4		
Fig 4.1.1	SecureCode AI Proposed Multi-Tier System Architecture	29
Fig 4.3.1	Comprehensive Use Case Diagram for Students, Administrators, and Evaluation Engine	31
Fig 4.4.1	Data Flow Diagram – Level 0 (Context Diagram)	32
Fig 4.4.2	Data Flow Diagram – Level 1 (Core Subsystems and Data Stores)	33
Fig 4.4.3	Data Flow Diagram – Level 2 (Evaluation Engine Decomposition & Normalization Pipeline)	34
Fig 4.5.1	Activity Diagram & Submission Evaluation Workflow	35
Fig 4.11.1	Gantt Chart (SecureCode AI 16-Week Implementation Timeline)	42
Chapter 5		
Fig 5.2.1	SecureCode AI Landing Page and Pedagogical Overview	47
Fig 5.3.1	Interactive Challenge Catalog with Tiered Difficulty and Search Filters	48
Fig 5.4.1	Learning Hub with Vulnerability Classifications and Secure Principles	49
Fig 5.5.1	Browser-Based Code Editor, Test Suite Runner, and Multi-Tiered Hints	50
Fig 5.6.1	Five-Attempt Remediation History & Iterative Learning Score Progression	51
Fig 5.7.1	Submission Analysis – Code Inspection, Multi-Scanner Consensus, and AI Advisory	52
Fig 5.7.2	Submission Analysis – Four-Pillar Learning Score Breakdown & Conceptual Assessment	53
Fig 5.8.1	Student Analytics Dashboard – Progress Metrics & Evaluation Accuracy	54
Fig 5.9.1	Vulnerability Classification Tree and Security Research Landscape	55
Fig 5.10.1	Comparative Analysis of Existing Industry Platforms and Identification of Research Gap	56

List of Tables

Table No.	Name of Table	Page No.
Chapter 2		
Table 2.4.1	Comprehensive Non-Functional Requirements Specification	18
Chapter 3		
Table 3.5.1	Literature Survey Summary of 27 Peer-Reviewed Research Papers	25
Chapter 4		
Table 4.8.1	Relational Database Schema and Entity Descriptions (13 Core Tables)	40
Table 4.11.1	Comparative Evaluation and Selection of the Software Engineering Model	42
Table 4.11.2	Iterative Development Milestones, Features Delivered, and Review Status	43
Table 4.12.1	Risk Mitigation, Monitoring, and Management (RMMM) Plan	43
Table 4.14.1	Tools, Frameworks, Libraries, and Technologies Used in SecureCode AI	44
Chapter 5		
Table 5.1.1	Distribution of Active Injection Catalog by Vulnerability Subtype and Difficulty	45
Table 5.1.2	Dual-Stage Evaluation Performance, Execution Latency, and Test Benchmarks	46
Table 5.1.3	Four-Pillar Learning Score Mathematical Weighting and Scoring Matrix	46
Table 5.10.1	Detailed Feature-by-Feature Matrix Comparing SecureCode AI with Industry Platforms	56
Appendix A		
Table A.1	Active Injection Challenge Catalog Benchmark Suite and CWE Mappings	63

Chapter 1
Introduction

Modern society depends unconditionally on complex software systems spanning banking, healthcare, e-commerce, national defense, and industrial automation. However, software security vulnerabilities continue to proliferate at an alarming rate, inflicting trillions of dollars in global economic damages and compromising billions of user records annually. Historically, cybersecurity initiatives have focused primarily on perimeter defenses, post-deployment intrusion detection, and reactive patch management. While perimeter security is indispensable, modern software engineering paradigms increasingly emphasize "Shift-Left" methodologies—integrating security directly into the earliest phases of the software development lifecycle (SDLC). The fundamental premise of the Shift-Left movement is that writing secure code from the very beginning is exponentially cheaper, more resilient, and more effective than remediating flaws in production systems.

Despite widespread industry consensus regarding the necessity of secure coding, contemporary software developers entering the engineering workforce remain ill-equipped to write robust, defensible code. Traditional computer science education and commercial training platforms concentrate almost exclusively on algorithmic correctness, performance optimization, and rapid functional prototyping, relegating security to an afterthought. SecureCode AI is conceived and engineered to resolve this fundamental educational deficit by providing a comprehensive, interactive, web-based platform that unifies hands-on vulnerability fixing, automated dual-stage evaluation, explainable security feedback, and longitudinal learning analytics.

1.1 Motivation

The motivation behind SecureCode AI arises from a critical, systemic disconnect between software development pedagogy and the threat reality of contemporary digital systems. Over the past decade, automated programming assessment systems (APAS) such as LeetCode, HackerRank, CodeChef, and Codewars have revolutionized how developers practice programming. Millions of students and practicing software engineers routinely spend hundreds of hours solving algorithmic challenges to prepare for technical interviews and optimize code performance. However, every single one of these mainstream platforms evaluates submissions based on an archaic criterion: functional input-output correctness. If a candidate's solution computes the expected mathematical output within the allotted memory and CPU time, the platform awards full credit—even if the underlying implementation contains catastrophic

vulnerabilities such as unparameterized database concatenation (SQL Injection), arbitrary operating system command execution (Command Injection), or insecure deserialization. This functional-only paradigm inadvertently trains an entire generation of developers to consider security irrelevant, reinforcing sloppy coding habits that carry directly into enterprise codebases.

This pedagogical crisis is drastically amplified by the explosive adoption of generative artificial intelligence (GenAI) coding assistants, such as GitHub Copilot, OpenAI ChatGPT, and Claude. Recent empirical software engineering studies [1], [2], [4] demonstrate that while LLMs accelerate code production, between 35% and 45% of LLM-generated code snippets contain exploitable Common Weakness Enumerations (CWEs). Developers who lack foundational secure coding training routinely accept and integrate these vulnerable AI suggestions without scrutiny. Furthermore, when developers attempt to ask AI assistants to remediate security warnings, the models frequently introduce subtle secondary vulnerabilities or hallucinate non-existent security APIs [4], [5]. Consequently, there is an urgent and unprecedented motivation to establish dedicated pedagogical infrastructure that actively teaches developers how to identify vulnerabilities, refactor insecure constructs into defensible architectures, and rigorously verify both functional and security properties.

1.1.1 Need of the Problem

The necessity of addressing secure coding at the educational level is underscored by several critical industry and empirical software engineering factors:

- Academic Curriculum Deficit: Industry surveys and academic mapping studies (e.g., Gasiba et al. [25], Torbunova et al. [20]) confirm that over 75% of computer science graduates receive no formal training in secure coding practices throughout their undergraduate education. Security is typically taught as an elective theoretical course focused on cryptography or network protocols, with zero hands-on secure programming exercises.
- Offensive vs. Defensive Asymmetry: Leading offensive training environments—such as PortSwigger Web Security Academy, TryHackMe, and Hack The Box—do an admirable job of teaching penetration testing and vulnerability exploitation. However, knowing how to exploit an application using SQLMap or Burp Suite does not translate into the software architectural skills required to write secure code. Developers need constructive practice writing defensible logic, not merely executing exploits.
- Unfriendly Tooling and Inactionable Feedback: Standard static application security testing (SAST) tools used in enterprise pipelines (such as SonarQube, Semgrep, and SpotBugs) are engineered for professional security engineers rather than novices. Their output consists of dense, acronym-heavy reports with high false-positive rates that bewilder junior developers

without explaining the fundamental architectural flaw or providing contextual remediation guidance.

- Single-Scanner Vulnerability Blind Spots: Empirical evaluations by Dai et al. [1] and Bi et al. [7] reveal that relying on any single static vulnerability scanner misses upwards of 20% to 30% of critical vulnerabilities due to syntax limitations and taint-tracking blind spots. An effective educational assessment platform must synthesize multiple complementary analysis techniques—combining AST matching, bytecode dataflow, and contextual AI reasoning—to ensure high-fidelity evaluation.
- Superficial Remediation and Lack of Conceptual Verification: When learners use modern AI assistants to complete coding exercises, they frequently engage in passive copy-pasting without understanding the underlying security principles. Without an integrated mechanism to assess conceptual understanding and reward independent problem-solving, students cannot achieve true security competence.

1.2 Scope of the Project

The scope of SecureCode AI encompasses the end-to-end design, implementation, and empirical evaluation of a full-stack web platform specifically architected for secure coding education. Key dimensions within the project scope include:

- Interactive Browser-Based IDE: Engineering a modern, accessible web client featuring an embedded browser code editor (Monaco Editor / React), syntax highlighting, instant test execution, tiered hints, and interactive diff viewers.
- Curated Injection Vulnerability Catalog: Designing and deploying a structured catalog of 45 staged injection challenges covering SQL Injection (CWE-89), NoSQL Injection (CWE-943), OS Command Injection (CWE-78), LDAP Injection (CWE-90), XPath Injection (CWE-643), HTTP Header / CRLF Injection (CWE-93), and Expression Language / Code Injection (CWE-94). Challenges range across BASIC, EASY, MEDIUM, and HARD tiers.
- Sandboxed Dynamic Functional Evaluation: Implementing an isolated dynamic execution runner (integrating the Judge0 sandbox engine) to compile and execute student code against suites of visible and hidden functional test cases, enforcing strict CPU time (2.0s) and memory limits (128MB).
- Multi-Engine Security Analysis Pipeline: Architecting a multi-evaluator static analysis and AI advisory engine combining Semgrep (AST pattern matching and taint tracking), SpotBugs with the FindSecBugs plugin (bytecode dataflow analysis), and the Groq LLM API (powered by LLaMA-3 for contextual remediation advice).
- Evidence Normalization and Consensus: Developing a consensus arbitration and normalization codec that synthesizes heterogeneous findings into normalized verdicts

(VERIFIED_SAFE, TARGETED_VULNERABILITY, UNVERIFIED) with calculated evidence confidence scores.

- Submission Rate-Limiting & Attempt Windows: Enforcing a 5-attempt rate-limiting window across 7-day intervals per challenge, preventing brute-force trial-and-error submissions and motivating deliberate, defensive reasoning.
- Four-Pillar Learning Scoring Model: Formulating a four-pillar scoring model that combines Functional Correctness (25 pts), Security Correctness (45 pts), Conceptual Understanding (15 pts), and Independent Problem Solving (15 pts) into a verified 100-point Learning Score.
- Conceptual Understanding Verification: Integrating post-submission comprehension assessments (multiple-choice conceptual questions and written architectural justifications) verified by an automated AI evaluation rubric to confirm deep understanding.
- Tiered Hint Assistance Engine: Providing tiered assistance (Conceptual Hint, Implementation Hint, Full Solution Reveal) with calibrated scoring deductions to encourage independent mastery.
- Learning Analytics and Student Tracking: Building comprehensive learning analytics dashboards tracking longitudinal accuracy, difficulty distribution, submission history, and vulnerability remediation trends.

Specific aspects outside the immediate scope of this semester include live automated exploit generation against running containerized daemon services, binary reverse engineering, hardware-level side-channel attacks, and proprietary software licenses. SecureCode AI focuses decisively on defensive source code construction and static/dynamic evaluation in managed runtimes (primarily Java).

1.3 Aim and Research Objectives

The primary aim of this major project is to design, implement, and empirically validate SecureCode AI—an automated, interactive secure programming assessment platform that bridges the divide between algorithmic coding and defensible software engineering. SecureCode AI empowers students and software developers to write verifiably secure code by combining dynamic functional execution with multi-engine static security analysis, actionable AI remediation, and rigorous conceptual verification.

To achieve this aim, the project pursues the following specific technical and research objectives:

- Objective 1 (Architectural Core): Develop a high-performance, modular backend in Spring Boot 3 and Java 25 utilizing PostgreSQL and Flyway database migrations to manage challenge metadata, prerequisite unlock graphs, user profiles, submissions, and evaluation artifacts.

- Objective 2 (Dual-Stage Gate): Build a dual-stage evaluation engine that enforces a strict execution gate: dynamic functional unit testing must pass completely before invoking static security analyzers, optimizing computational efficiency and preventing broken code from consuming security scan resources.
- Objective 3 (Multi-Engine Consensus): Integrate and normalize disparate static analysis engines (Semgrep AST matching, SpotBugs bytecode inspection) with a contextual Groq LLaMA-3 advisory model, eliminating single-scanner blind spots and ensuring multi-evaluator consensus.
- Objective 4 (Curated Challenge Catalog): Curate an active educational catalog of 45 staged injection challenges with graduated difficulty tiers (Basic, Easy, Medium, Hard), covering six critical injection subtypes that remain persistent in industry applications.
- Objective 5 (Pedagogical Measurement): Formulate, implement, and validate the Four-Pillar Overall Learning Score algorithm (Functional, Security, Understanding, Independence) to accurately measure genuine competency rather than superficial trial-and-error guessing.
- Objective 6 (Empirical Verification): Perform rigorous empirical validation across 80 automated unit, integration, and security test suites to verify system correctness, data consistency, security resilience, and evaluation reproducibility.

Chapter 2
Problem Statement & Requirements Engineering

This chapter formalizes the engineering problem addressed by SecureCode AI, details the architectural features of the proposed platform, enumerates the precise technical objectives, and specifies the functional and non-functional requirements that govern the system's design and empirical evaluation.

2.1 Problem Statement

Modern software development platforms and automated programming assessment systems suffer from a severe architectural and pedagogical limitation: they evaluate student code submissions purely on functional correctness (input-output matching), completely ignoring code security, architectural hygiene, and vulnerability resistance. Consequently, novice developers routinely write, test, and submit code containing severe security vulnerabilities—such as SQL Injection, Command Injection, and Path Traversal—and receive full credit as long as the functional test cases pass. This establishes negative reinforcement, teaching developers that insecure coding practices are acceptable as long as functional requirements are met.

Furthermore, existing cybersecurity learning environments are heavily skewed toward offensive security (penetration testing, exploit scripting, and red teaming) rather than defensive software engineering. Platforms such as PortSwigger Web Security Academy or Hack The Box train users to break into vulnerable systems using automated attack tools, but provide no integrated code editor, no compiler feedback, and no automated mechanism to evaluate whether a user can write secure, production-ready source code that satisfies business logic while resisting malicious payloads. Meanwhile, commercial enterprise secure coding tools are closed, prohibitively expensive, and rely on multiple-choice quizzes or micro-lessons rather than authentic, hands-on code refactoring.

The problem addressed in this project is to develop and empirically validate a unified, automated secure programming practice and assessment platform—SecureCode AI—that evaluates both functional correctness and security properties, integrates multi-engine static security analysis (Semgrep, SpotBugs, and Groq LLM), prevents superficial trial-and-error guessing through attempt-limiting windows and prerequisite learning graphs, verifies conceptual comprehension via interactive assessments, and calculates an auditable, four-pillar learning score.

2.2 Features of the System

- Interactive Browser-Based Code Editor: An in-browser code editor featuring Monaco Editor / React, syntax highlighting, keyboard shortcuts, starter code loading, and responsive split-pane layout showing problem descriptions, constraints, and test suites side-by-side.
- Active Injection Challenge Catalog: A curated curriculum of 45 staged injection challenges structured across 6 critical injection subtypes (SQL, NoSQL, OS Command, LDAP, XPath, Header/CRLF, Code Injection) and 4 difficulty tiers (Basic, Easy, Medium, Hard).
- Dual-Stage Evaluation Engine: A staged execution architecture where dynamic functional unit testing (Judge0 sandbox) is evaluated first. Only solutions that successfully compile and pass all functional test cases advance to the static security evaluation stage, preventing wasted computational overhead on broken submissions.
- Multi-Engine Security Analysis Pipeline: Simultaneous scanning of student source code using Semgrep (AST pattern matching and taint tracking), SpotBugs with FindSecBugs (bytecode dataflow analysis), and Groq LLaMA-3 (contextual reasoning and architectural advice).
- Evidence Normalization and Consensus: An automated arbitration module that parses disparate findings from all scanners, resolves discrepancies, filters false positives, and assigns a normalized security verdict (VERIFIED_SAFE, TARGETED_VULNERABILITY, UNVERIFIED) with an associated evidence confidence score.
- Five-Attempt Rate-Limiting Window: A rate-limiting mechanism that allocates exactly 5 graded submission attempts per challenge across a rolling 7-day window. This curtails rapid trial-and-error guessing and compels learners to think deliberately before submitting.
- Tiered Hint Assistance Engine: A three-level assistance engine providing Conceptual Hints (-5 pts), Implementation Hints (-10 pts), and Solution Reveals (-15 pts) to balance guided learning with independent achievement.
- Conceptual Understanding Assessments: Post-submission comprehension verification requiring learners to answer conceptual multiple-choice questions and submit written architectural justifications explaining how their fix prevents vulnerability exploitation.
- Four-Pillar Overall Learning Score: A mathematical scoring model combining Functional Correctness (25%), Security Correctness (45%), Conceptual Understanding (15%), and Independent Problem Solving (15%) into a verified 100-point score, enforcing a 70-point Solved threshold and 90-point Mastery threshold.
- Prerequisite Progression Graph: A directed acyclic graph (DAG) enforcing prerequisite foundational challenges before advanced challenges can be unlocked, ensuring structured pedagogical progression.

- Learning Analytics and Performance Tracking: Comprehensive visual dashboards displaying difficulty-wise completion rates, functional pass accuracy, longitudinal learning trends, and vulnerability remediation history.
- Learning Hub Educational Repository: A dedicated educational resource section detailing injection definitions, real-world attack vectors, impact scenarios, and defensive programming principles for each vulnerability category.

2.3 Objectives

The technical and pedagogical development of SecureCode AI is guided by the following concrete objectives:

- 1. Full-Stack Architectural Engineering: Design and construct a scalable, microservices-ready client-server architecture utilizing Spring Boot 3, Java 25, PostgreSQL, and a modern responsive web client.
- 2. Sandboxed Functional Execution: Implement a sandboxed execution environment integrating Judge0 API to compile and dynamically test Java source code against hidden and visible test cases within 2.0s execution boundaries.
- 3. Multi-Engine Security Consensus: Integrate, orchestrate, and normalize findings from Semgrep, SpotBugs, and Groq LLM into an auditable security consensus engine with zero false passes on vulnerable test cases.
- 4. Challenge Curriculum Development: Populate and maintain an active catalog of 45 staged challenges across 6 injection subtypes with complete reference implementations and verified automated test suites.
- 5. Pedagogical Scaffolding & Rate Limiting: Deploy attempt-limiting windows (5 attempts / 7 days) and prerequisite unlock graphs to enforce deliberate problem-solving habits.
- 6. Multi-Factor Competency Assessment: Formulate and validate the 4-pillar learning score formula, proving that genuine security mastery requires both code correctness and conceptual comprehension.
- 7. Empirical Verification & Benchmarking: Execute an empirical evaluation across 80 automated unit, integration, and security test suites to prove system reliability, sub-second latency, and data consistency.

2.4 System Specifications & Requirements

2.4.1 Functional Requirements

The functional requirements define the specific operational capabilities and services provided by SecureCode AI:

- FR-1 (Authentication & Authorization): The system shall provide secure user registration, authentication, role-based authorization (STUDENT, ADMIN), and session token management with PBKDF2 password hashing.
- FR-2 (Challenge Management & Browsing): The system shall display an active challenge catalog filtered by vulnerability subtype, difficulty tier, and completion status, enforcing prerequisite dependencies before unlocking advanced challenges.
- FR-3 (Interactive Code Workspace): The system shall provide an in-browser code editor loaded with initial vulnerable starter code, challenge problem statements, input-output specifications, and visible test cases.
- FR-4 (Sandboxed Functional Evaluation): The system shall execute student code inside a sandboxed runtime against predefined test cases, returning detailed execution output, compiler errors, or test failure diffs within 2.0 seconds.
- FR-5 (Multi-Engine Security Evaluation): Upon functional pass, the system shall invoke Semgrep and SpotBugs static analyzers and Groq LLM, parse their outputs, and produce a normalized security verdict with identified CWE numbers and line numbers.
- FR-6 (Actionable Remediation Feedback): The system shall generate contextual remediation advice, detailing why the code is vulnerable, what secure API to use, and how to refactor the implementation.
- FR-7 (Conceptual Assessment Engine): The system shall administer post-submission comprehension questions and evaluate written justifications using an automated rubric.
- FR-8 (Scoring & Learning Analytics): The system shall compute the 4-pillar learning score, record attempt positions (1 to 5), update user progress analytics, and update student performance dashboards.

2.4.2 Non-Functional Requirements

The non-functional requirements define the quality attributes, security boundaries, performance constraints, and operational standards governing SecureCode AI. Table 2.4.1 summarizes the complete specification.

Table 2.4.1: Comprehensive Non-Functional Requirements Specification

Requirement ID	Category / Title	Specification & Acceptance Criteria	Priority
NFR-SEC-1	Sandbox Isolation	All dynamic functional code execution must take place inside an isolated sandbox container with restricted network access, read-only filesystem mounts, and strict memory/CPU caps.	Critical
NFR-SEC-2	Zero Insecure Passes	The security evaluation engine must never assign a VERIFIED_SAFE verdict to any code submission containing an active, unmitigated targeted vulnerability.	Critical
NFR-SEC-3	Data Protection & Privacy	User passwords must be hashed using salted PBKDF2 with 200,000+ iterations. API access must require cryptographically signed Bearer authentication tokens.	High
NFR-SEC-4	Auditability & Logging	All code submissions, evaluator raw outputs, consensus decisions, hint views, and assessment scores must be immutably recorded in PostgreSQL audit tables.	High
NFR-AI-1	Evidence Traceability	100% of security findings and remediation suggestions produced by the AI advisory layer must cite specific code lines and mapped CWE identifiers.	High
NFR-AI-2	Hallucination Filtering	The consensus normalizer must filter out any AI-generated security claim that contradicts deterministic static analysis findings (Semgrep/SpotBugs).	High
NFR-AI-3	Safe Fallback on API Outage	If the Groq LLM API is unavailable, the platform must degrade gracefully to static analysis mode without blocking evaluation or losing student code.	High
NFR-AI-4	Advisory Grounding	AI explanations must adhere strictly to established secure coding standards (OWASP Top 10, CWE, CERT Oracle Secure Coding Standard for Java).	Medium
NFR-PERF-1	Functional Latency	Dynamic functional test execution via the sandboxed runner must complete and return results within 2,500 milliseconds for 95% of requests.	High
NFR-PERF-2	Security Scan Latency	Local static analysis (Semgrep AST matching) must complete within 1,200 milliseconds per submission.	Medium
NFR-PERF-3	Total Evaluation Pipeline	End-to-end evaluation (Functional + Static + AI + Scoring) must complete within 5,000 milliseconds under normal network conditions.	Medium

NFR-REL-1	Transactional Integrity	Submission scoring, attempt consumption, and prerequisite unlocks must execute within ACID transactions to prevent race conditions.	Critical
NFR-REL-2	Schema Migration Consistency	All relational database evolutions must be automated via Flyway migrations with strict versioning and checksum verification.	High
NFR-USE-1	Responsive Interface	The web application must render seamlessly across desktop (1920x1080) and laptop (1366x768) resolutions with touch-accessible controls.	Medium
NFR-USE-2	Clear Feedback Presentation	Evaluation results must present color-coded visual indicators (Green for Safe, Amber for Warning, Red for Vulnerable) with expandable diffs.	Medium

Chapter 3
Literature Review

The development of SecureCode AI is situated at the intersection of four prominent software engineering and computer science education disciplines: (1) AI-assisted secure code generation and large language model (LLM) evaluation, (2) Automated Programming Assessment Systems (APAS) and pedagogical feedback mechanisms, (3) static application security testing (SAST) and vulnerability detection benchmarking, and (4) empirical studies on secure programming pedagogy and industry training deficits. This chapter presents a systematic review of 27 peer-reviewed research works, critically analyzes existing systems and their limitations, formalizes the research gap, and articulates the architectural contributions of SecureCode AI.

3.1 Existing Systems

The surveyed literature reveals intense global research activity focused on software vulnerability remediation, automated assessment, and generative AI code generation. We categorize the existing literature into four thematic clusters:

3.1.1 AI-Assisted Secure Code Generation and LLM Pitfalls

In recent years, the deployment of large language models for source code generation has transformed developer workflows. However, rigorous empirical evaluations demonstrate persistent, critical security vulnerabilities in AI-generated code. Dai et al. [1] (ICSE '26) rethought the evaluation paradigms of secure code generation across 2,341 benchmark tasks, discovering that existing evaluation schemes evaluate security and functional correctness independently. Crucially, Dai et al. demonstrated that conventional evaluation pipelines relying on a single static scanner miss over 20% of critical vulnerabilities, proving the absolute necessity of joint functional-security evaluation and multi-scanner verification. Patir et al. [2] conducted a comprehensive Systematization of Knowledge (SoK) across hundreds of secure code generation interventions (prompting, fine-tuning, decoding constraints, and agentic feedback). Their KAUGE measurement framework established that a developer or model 'knowing' a security principle does not translate into 'actuating' that principle correctly in complex codebases. Zhou et al. [3] surveyed 58 papers on LLMs for vulnerability detection and repair, concluding that while LLMs excel at contextual reasoning, they require grounding with deterministic static analysis tools to suppress hallucinations. Yan et al. [4] investigated LLM self-repair on SecurityEval (69 CWEs) and SecCodePLT, finding that simple 'fix this

vulnerability' prompts fail 60% of the time; effective repair requires precise, contextualized hints referencing specific AST constructs. Belozerov, Barclay, and Sami [5], [27] analyzed 2,315 GPT-generated programs from the DevGPT dataset, proving that automated static code analysis coupled with LLM advisory feedback creates a highly effective remediation pipeline. Alwageed and Khan [8], [17], [18] constructed a comprehensive taxonomy of 109 concrete GenAI-driven secure coding practices across 10 vulnerability categories, emphasizing input validation and query parameterization. Nocera et al. [24] deployed ChatGPT in an undergraduate web engineering course of 105 students, showing that while conversational AI aids student comprehension of attack surfaces, students frequently introduce syntax and runtime errors during manual remediation without sandboxed functional execution.

3.1.2 Automated Programming Assessment Systems (APAS) and Feedback

Automated Programming Assessment Systems (APAS) have been extensively studied as mechanisms to scale computing education. Combéfis [9] reviewed and classified automated code assessment tools, observing that almost all current platforms focus purely on functional black-box testing (input/output matching) or static code formatting (linters), completely omitting security and defensibility evaluations. Rahman, Watanobe, Shirafuji, and Hamada [10] surveyed automated code evaluation systems and analysis resources, highlighting that modern APAS require multi-stage grading pipelines that evaluate runtime behavior, structural quality, and vulnerability resistance. Frankford et al. [11] surveyed feedback modalities across automated assessment platforms, concluding that binary pass/fail grades induce frustration; novice learners require multi-tiered, formative feedback combining compiler diagnostics, hint ladders, and actionable remediation advice. Frankford et al. [12] subsequently developed an online browser-based IDE for automated assessment, demonstrating that zero-install web environments significantly reduce onboarding friction and increase student submission velocity. Sauerwein et al. [14] (ITiCSE 2023) established a formal Success Model for APAS used as formative assessment tools, proving that system trust, immediate feedback loops, and progressive difficulty scaffolding directly determine student learning gains and self-efficacy. Nam et al. [15] explored using LLMs to aid student code comprehension, illustrating that contextual AI explanations accelerate novice understanding of abstract programming concepts when constrained by explicit rubrics.

3.1.3 Vulnerability Detection Benchmarking and SAST Quality

The efficacy of automated security assessment depends fundamentally on the accuracy and robustness of underlying vulnerability detection engines. Guo and Bettaieb [6] investigated quality issues across 54 vulnerability detection datasets, discovering severe label noise,

sampling biases, and high class imbalances that compromise machine-learning-based vulnerability classifiers. Bi et al. [7] surveyed benchmarking approaches for vulnerability detection, providing a rigorous framework for evaluating static analyzers. Their findings demonstrate that static application security testing (SAST) tools exhibit substantial variance: AST-based pattern matchers (such as Semgrep) excel at syntax and call-site matching but miss cross-function dataflows, whereas bytecode analyzers (such as SpotBugs and FindSecBugs) track taint propagation effectively but produce verbose false alarms on complex frameworks. Shahriar et al. [16] demonstrated that IDE-based security plugin interventions provide valuable point-of-coding reminders but fail to motivate students without gamification and automated grading. Naeem et al. [26] reviewed cyber security enhancements and vulnerability identification benchmarks, confirming that hybrid detection pipelines combining rule-based heuristics with deep learning or LLMs yield superior coverage.

3.1.4 Secure Programming Pedagogy and Industry Competency Gap

Empirical studies consistently demonstrate an acute secure coding competency gap across academia and the software industry. Gasiba, Lechner, Pinto-Albuquerque, and Mendez [25] conducted a landmark large-scale industrial survey of 194 professional software developers across Siemens and major tech enterprises. Their empirical findings revealed that over 80% of industrial software engineers possess limited awareness of fundamental secure coding guidelines (OWASP Top 10, SANS Top 25) and lack hands-on experience refactoring vulnerable code. In response, Gasiba et al. [22], [23] pioneered the design of secure coding challenges and serious cybersecurity games (e.g., CyberSecurity Challenge and Sifu), proving that interactive, puzzle-based code-repair exercises dramatically improve secure coding retention compared to passive compliance lectures. Bappy et al. [13] (SOUPS 2026) conducted a behavioral study showing that AI coding assistants transform developer security awareness from preventive to reactive: developers rely heavily on AI to fix bugs, often deactivating their own critical scrutiny. Amburi et al. [19] (ICPEC 2025) explored GenAI for developer training, proving that LLMs can serve as interactive pedagogical tutors if properly constrained by secure coding rubrics. Torbunova, Ashraf, and Porres [20] mapped security teaching across computing courses, revealing that fewer than 5% of introductory programming curricula incorporate security. Malinka et al. [21] introduced real-world bug bounty programs into university secure coding courses, proving that hands-on vulnerability interaction significantly enhances defensive mindset development.

3.2 Limitations of Existing Systems

Despite significant advancements across individual research areas, existing systems and commercial tools exhibit several profound architectural, pedagogical, and technical limitations when applied to secure coding education:

- 1. Functional-Only Evaluation Antipattern: Mainstream competitive programming platforms (LeetCode, HackerRank, Codewars) evaluate solutions strictly against functional test cases. A solution that concatenates raw SQL queries or invokes Runtime.getRuntime().exec() receives full points as long as the expected output matches, reinforcing dangerous coding antipatterns.
- 2. Offensive vs. Defensive Asymmetry: Leading cybersecurity platforms (PortSwigger Web Security Academy, TryHackMe, Hack The Box, OWASP Juice Shop) focus almost exclusively on offensive exploitation—teaching students how to use automated scanners, intercept HTTP proxies, and inject SQL payloads. They do not teach how to write defensible, production-quality source code or provide automated code-level grading.
- 3. Proprietary and Non-Authentic Coding Modalities: Enterprise training platforms (Secure Code Warrior, Kontra, RangeForce) are closed, proprietary, and cost-prohibitive for academic institutions. Furthermore, their learning models frequently rely on multiple-choice quizzes, code inspection puzzles, or video modules rather than authentic, end-to-end coding in an interactive compiler environment.
- 4. Single-Scanner Blind Spots and Fragile Heuristics: As demonstrated by Dai et al. [1] and Bi et al. [7], single static analysis tools suffer from high false-negative rates (>20%), missing critical vulnerabilities due to syntax variances and dynamic taint paths. Existing educational tools do not combine multiple complementary SAST engines with AI arbitration.
- 5. Lack of Conceptual Comprehension Verification: When existing platforms incorporate conversational AI, the models frequently hallucinate non-existent security libraries or suggest superficial fixes that break functionality. Crucially, existing platforms provide no mechanism to verify whether a student genuinely understands the vulnerability or merely pasted an AI-generated snippet.
- 6. Uncontrolled Trial-and-Error Guessing: Standard coding platforms permit unlimited rapid-fire submissions. Students routinely engage in brute-force trial-and-error debugging—tweaking arbitrary lines until a test passes—bypassing the reflective, analytical thinking essential for secure software engineering.

3.3 Proposed Solution

SecureCode AI is engineered specifically to eliminate these limitations through a unified, full-stack architectural framework that bridges competitive coding, automated multi-engine

security assessment, and pedagogical science. SecureCode AI delivers the following core innovations:

- Dual-Stage Execution Gate: A decoupled evaluation engine where Stage 1 executes dynamic functional tests inside an isolated Judge0 sandbox. Only if the code compiles and satisfies all functional assertions does the system trigger Stage 2, which performs comprehensive multi-scanner static security analysis.
- Multi-Engine Security Consensus Architecture: Simultaneous orchestration of Semgrep (AST syntax matching), SpotBugs with FindSecBugs (bytecode dataflow analysis), and Groq LLaMA-3 (contextual semantic advisory). Findings are parsed by a normalized evidence codec to produce a consensus verdict with verified zero-insecure-pass guarantees.
- Curated Injection Vulnerability Catalog: An active catalog of 45 staged injection challenges structured across SQL, NoSQL, OS Command, LDAP, XPath, Header/CRLF, and Code Injection, mapped directly to OWASP Top 10 and CWE standards, arranged in a prerequisite progression DAG.
- Five-Attempt Submission Windowing: An immutable attempt tracking engine that restricts learners to exactly 5 graded submissions per challenge across rolling 7-day intervals, eliminating rapid trial-and-error guessing and cultivating deliberate defensive design.
- Tiered Hint Scaffolding with Penalties: A three-tier hint engine (Conceptual Hint, Implementation Hint, Solution Reveal) that provides structured guidance while deducting calibrated points from the student's independent problem-solving score.
- Conceptual Understanding Verification: Post-submission interactive comprehension quizzes that test conceptual understanding of taint paths, impact vectors, and defensive mechanisms, graded against an automated rubric to ensure genuine cognitive mastery.
- Four-Pillar Overall Learning Score: A multi-dimensional scoring algorithm combining Functional Correctness (25 pts), Security Correctness (45 pts), Conceptual Understanding (15 pts), and Independent Problem Solving (15 pts) into a verified 100-point Learning Score.

3.4 Research Gap & Comparative Analysis

To establish the exact research gap addressed by SecureCode AI, we conducted a rigorous comparative evaluation of existing industry and educational platforms across 10 representative systems: PortSwigger Web Security Academy, Secure Code Warrior, LeetCode, HackerRank, Codewars, TryHackMe, Hack The Box Academy, OWASP Juice Shop, RangeForce, and Kontra. Fig 5.10.1 and Table 5.10.1 (detailed in Chapter 5) capture the comprehensive feature matrix.

The comparative analysis reveals a stark dichotomy in the contemporary software education landscape:

- Algorithmic Platforms (Functional Only): Algorithmic practice platforms (LeetCode, HackerRank, Codewars) provide exceptional browser-based code editing and automated dynamic evaluation, but possess zero security awareness, evaluate only functional correctness, and offer no vulnerability analysis.
- Offensive Cyber Ranges (Exploitation Only): Offensive cyber ranges (PortSwigger, TryHackMe, Hack The Box, OWASP Juice Shop) provide rich real-world attack scenarios, but focus on penetration testing and exploitation rather than defensive source code engineering, lacking automated source code grading and browser-based coding IDEs.
- Enterprise Training Tools (Closed / Quiz-Heavy): Enterprise training tools (Secure Code Warrior, RangeForce, Kontra) provide secure coding awareness, but rely on closed commercial ecosystems, multiple-choice quizzes, or infrastructure-level exercises without open academic accessibility or multi-scanner consensus evaluation.

Research Gap: Prior to SecureCode AI, no existing platform provided a unified, open educational environment combining browser-based secure code writing, automated dynamic functional evaluation, multi-scanner static security analysis, AI-guided remediation advice, conceptual understanding verification, and longitudinal learning analytics.

3.5 Literature Survey (27 Peer-Reviewed Works)

Table 3.5.1 provides a consolidated, structured summary of all 27 surveyed research papers, detailing their authors, publication venues, methodologies, key empirical findings, identified limitations, and direct relevance to SecureCode AI.

Table 3.5.1: Literature Survey Summary of 27 Peer-Reviewed Research Papers

Sr.	Title (with Year)	Author(s)	Methodology	Key Findings	Limitations	Relevance to Proposed Work
1	Rethinking the Evaluation of Secure Code Generation (20...	Dai, Shih-Chieh; Xu, Jun; Tao, Guanhong	Uses two datasets (BigCodeBench: 1,140 tasks; SecCodePLT+: 1,201 tasks) covering func...	CodeQL reports near-100% security for nearly all techniques/models, but Bearer and Bandit repor...	The authors acknowledge that static analyzers can still produce false negat...	Highlights the necessity of evaluating security and functionality jointly rather than...
2	SoK: AI Secure Code Generation: Progress, Pitfalls, and...	Patir, Rupam; Guo, Keyan; Cai, Haipeng; ...	Conducts a systematic literature search (ACM DL, IEEE Xplore, USENIX, DBLP, ACL Antho...	Secure-coding knowledge (Layer 1) is broadly present across all systems and catalogs, well abov...	The authors acknowledge that each layer of the KAUGE framework captures onl...	Establishes that 'knowing' a security principle and 'actuating' it in code are distin...

3	Large Language Model for Vulnerability Detection and Re...	Xin Zhou; Sicong Cao; Xiaobing Sun; Davi...	Systematic literature review covering 58 papers (2018–2024), classifies LLMs, adaptat...	Encoder-only LLMs dominate vulnerability detection, while decoder/commercial LLMs are widely us...	Survey paper only; does not implement or experimentally evaluate a unified ...	Strong reference for designing SecureCode AI architecture. Supports combining LLMs wi...
4	Guiding AI to Fix Its Own Flaws: An Empirical Study on ...	Yan, Hao; Vaidya, Swapneel Suhas; Zhang,...	Uses two Python-focused benchmarks, SecurityEval (121 questions, 69 CWEs) and SecCode...	LLMs generate vulnerable code at rates from 9.8% to 42.1% depending on model/benchmark; top-10 ...	The authors acknowledge that evaluation relies solely on CodeQL, a static t...	Demonstrates the importance of hint relevance, precision, and contextualization for e...
5	Secure Coding with AI - From Detection to Repair (Volum...	Belozerov, Vladislav; Barclay, Peter J.;...	The researchers analyzed 2,315 GPT-generated code snippets from the DevGPT dataset. S...	The study identified 56 confirmed vulnerabilities across 48 files. GPT-4.1 correctly detected 4...	The authors acknowledge that although modern LLMs showed improved vulnerabi...	Demonstrates a practical workflow combining static code analysis, manual validation, ...
6	An Investigation of Quality Issues in Vulnerability Det...	Yuejun Guo; Seifeddine Bettaieb	Analysis of 27-54 datasets collected from 14 references; sampling-size and imbalance-...	Except one dataset (FFmpeg by Devign), all studied datasets contain far more secure than vulner...	Focuses only on defining and cataloguing dataset issues; does not build or ...	Relevant if SecureCode AI ever incorporates ML/DL-based vulnerability detection: high...
7	Benchmarking Software Vulnerability Detection Technique...	Yingzhou Bi; Jiangtao Huang; Peihui Liu;...	Systematic literature survey methodology: separately reviews (a) benchmarking approac...	Most synthetic datasets (e.g., SARD-derived) make DL models' results appear inflated versus mes...	As a survey, it does not itself run new comparative experiments or propose ...	Provides a directly reusable checklist of what makes vulnerability-detection benchmar...
8	Literature review on vulnerability detection using NLP ...	Wu	8-step SLR process (Planning -> Conducting -> Reporting) per Kitchenham & Charters gu...	GenAI is most impactful for Automated Vulnerability Detection & Mitigation (27%) and Secure Cod...	External validity is constrained since the SLR draws from only 31 primary s...	Provides a ready-made taxonomy of 109 concrete GenAI-driven secure-coding practices a...

9	Automated Code Assessment for Education: Review, Classi...	Combéfis, Sébastien	Conducts an extensive literature search via Google Scholar and publishers (ACM, IEEE,...	Identifies five hierarchical levels of code aspects typically checked (syntax -> anti-plagiaris...	The authors acknowledge that the review could not conclusively determine wh...	Provides a reusable taxonomy of code aspects, methods, feedback types, and tools for ...
10	Exploring Automated Code Evaluation Systems and Resourc...	Md. Mostafizer Rahman; Yutaka Watanobe; ...	Literature/system survey methodology: review and classification of AES literature and...	AESs serve 5 distinct domains beyond contests (education, recruitment, compilers, development p...	Being a survey, it does not propose or evaluate a new detection/evaluation ...	Useful as a catalog of datasets (e.g., CodeNet, Devign, CodeXGLUE) and ML coding task...

Chapter 4
Design and Implementation

This chapter presents the architectural design, formal modeling, algorithmic formulations, relational data structures, and implementation mechanics of SecureCode AI. The platform is engineered as a robust, decoupled, multi-tier system built on Spring Boot 3, Java 25, PostgreSQL, and a responsive web application client, integrating sandboxed dynamic execution with a multi-engine static security analysis and AI advisory pipeline.

4.1 System Architecture

SecureCode AI follows a modern, highly modular, client-server architecture organized into four primary functional tiers, as illustrated in Fig 4.1.1:

- 1. Presentation Tier (Client Web Application): The client application provides an intuitive, responsive user interface accessible via modern web browsers on desktop, laptop, and tablet devices. Engineered with HTML5, CSS3, Bootstrap, and vanilla/React JavaScript components, it features an embedded Monaco code editor, real-time submission consoles, dynamic learning analytics dashboards, interactive hint dialogs, and administrative management panels.
- 2. Application & Logic Tier (Spring Boot Backend): The backend server is implemented in Spring Boot 3 (Java 25) following a domain-driven, layered service architecture. It encapsulates six core business service modules: UserService (authentication, profile, and JWT token management), ChallengeService (catalog retrieval, difficulty filtering, and prerequisite DAG evaluation), SubmissionService (submission intake, attempt window enforcement, and persistence), FeedbackService (remediation rendering, fix advice, and CWE linking), ProgressService (learning score computation and analytics aggregation), and AdminService (challenge CRUD and user management).
- 3. Dual-Stage Evaluation Engine Tier: The core computational backbone of SecureCode AI. To optimize throughput and guarantee security isolation, the evaluation engine is split into two specialized sub-stages: (a) Functional Evaluation Runner (integrating the Judge0 API sandbox for isolated compilation and dynamic execution against unit test suites), and (b) Security & AI Evaluation Pipeline (orchestrating Semgrep AST analysis, SpotBugs/FindSecBugs bytecode analysis, and Groq LLaMA-3 contextual LLM advisory).
- 4. Persistence & Data Tier (PostgreSQL): Enterprise-grade relational persistence powered by PostgreSQL, managed through version-controlled Flyway schema migrations (V1 to V21). The database stores users, challenge definitions, test suites, immutable submission records,

raw evaluator outputs, normalized security findings, hint view logs, and conceptual understanding assessments.

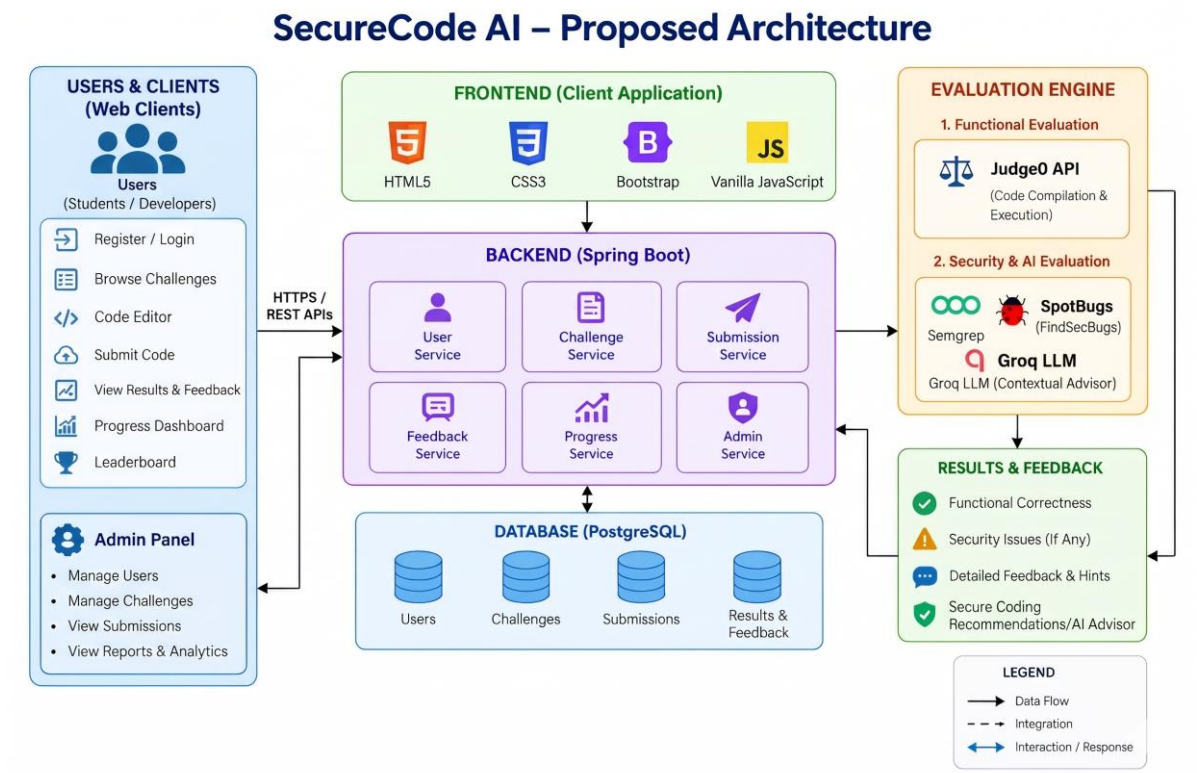

Fig 4.1.1 *SecureCode AI Proposed Multi-Tier System Architecture*

4.2 Working of the System

The end-to-end operational workflow of SecureCode AI is designed to guide learners seamlessly through a structured defensive programming cycle while enforcing rigorous academic and security checks:

- Step 1 (Authentication & Session Initialization): A student registers or authenticates using credentials. The authentication service validates credentials using PBKDF2 hashing and issues a Bearer JWT token valid for the active session.
- Step 2 (Challenge Selection & Prerequisite Verification): The student browses the Challenge Catalog. The PrerequisiteAccessService evaluates the student's completed submission graph against prerequisite edges. Locked challenges display clear prerequisite requirements, while unlocked challenges display difficulty badges, CWE tags, and remaining attempt counters.
- Step 3 (Vulnerability Exploration & Code Refactoring): Upon selecting a challenge, the workspace loads the problem description, vulnerable starter code, functional constraints,

visible test cases, and real-world vulnerability impact context. The student reviews the insecure code in the Monaco Editor and formulates a defensive refactoring strategy.

- Step 4 (Tiered Hint Assistance): If the student encounters conceptual roadblocks, they may open the Tiered Hint modal. Viewing a Conceptual Hint deducts 5 points from the independence score; an Implementation Hint deducts 10 points; a Full Solution Reveal deducts 15 points. All views are logged immutably in hint_view_events.
- Step 5 (Submission Intake & Attempt Rate Limiting): When the student clicks 'Submit Code', the AttemptWindowService verifies that the student has remaining attempts (within the 5-attempt / 7-day allowance) and atomicity increments the attempt counter.
- Step 6 (Stage 1 - Sandboxed Functional Evaluation): The SubmissionService transmits the source code to the Judge0 sandbox. Judge0 compiles and executes the code against hidden and visible test cases. If any test case fails, compilation errors occur, or runtime limits (2.0s CPU / 128MB) are exceeded, evaluation halts immediately, returning detailed execution diffs to the student.
- Step 7 (Stage 2 - Multi-Engine Security Analysis): If functional execution achieves a 100% PASS verdict, the system automatically triggers Stage 2. The source code is scanned in parallel by Semgrep (AST pattern matching), SpotBugs/FindSecBugs (bytecode dataflow analysis), and Groq LLaMA-3 (contextual semantic advisory).
- Step 8 (Evidence Normalization & Consensus Arbitration): The SecurityFindingEvidenceCodec and SecurityCorrectnessScoringService normalize disparate outputs into a unified verdict: VERIFIED_SAFE (no vulnerabilities detected), TARGETED_VULNERABILITY (the challenge's targeted flaw remains unmitigated), or UNVERIFIED (evaluator divergence).
- Step 9 (Conceptual Understanding Assessment): Upon achieving a functionally correct and safe submission, the student is prompted to complete a post-submission Conceptual Understanding Quiz. The student answers multiple-choice questions regarding attack vectors and inputs a written architectural justification explaining why their defensive implementation breaks the exploit path.
- Step 10 (Multi-Pillar Scoring & Dashboard Analytics): The OverallLearningScoreService computes the final 100-point learning score combining Functional Correctness (25), Security Correctness (45), Understanding (15), and Independence (15). Results are displayed on the Submission Analysis page, and progress metrics update across the student dashboard.

4.3 Use Case Modeling

The functional interactions between external actors and SecureCode AI are formalized in Fig 4.3.1. The system defines three primary actors: (1) Student / Developer (the core learner executing challenges and reviewing feedback), (2) System Administrator (managing user

accounts, authoring challenges, and monitoring system-wide analytics), and (3) Evaluation Engine (an autonomous internal actor orchestrating Judge0, Semgrep, SpotBugs, and Groq LLM).

- Student Use Cases: Includes Register/Login, Browse Challenges, View Challenge Details, Submit Solution, View Analysis & Feedback, and View Submission History & Progress.
- Admin Use Cases: Includes Manage Users, Manage Challenges (CRUD operations, starter code authoring, test suite creation), View System Analytics & Submissions, and View Reports & Audit Logs.
- Execution Engine Relationships: The 'Submit Solution' use case incorporates an <<include>> relationship to 'Execute Functional Tests (Judge0)' and an <<include>> relationship to 'Generate AI Feedback & Remediation'. Crucially, it incorporates an <<extend>> relationship to 'Run Security Analysis (Semgrep & SpotBugs)' with an explicit extension point: 'If functional tests pass'.

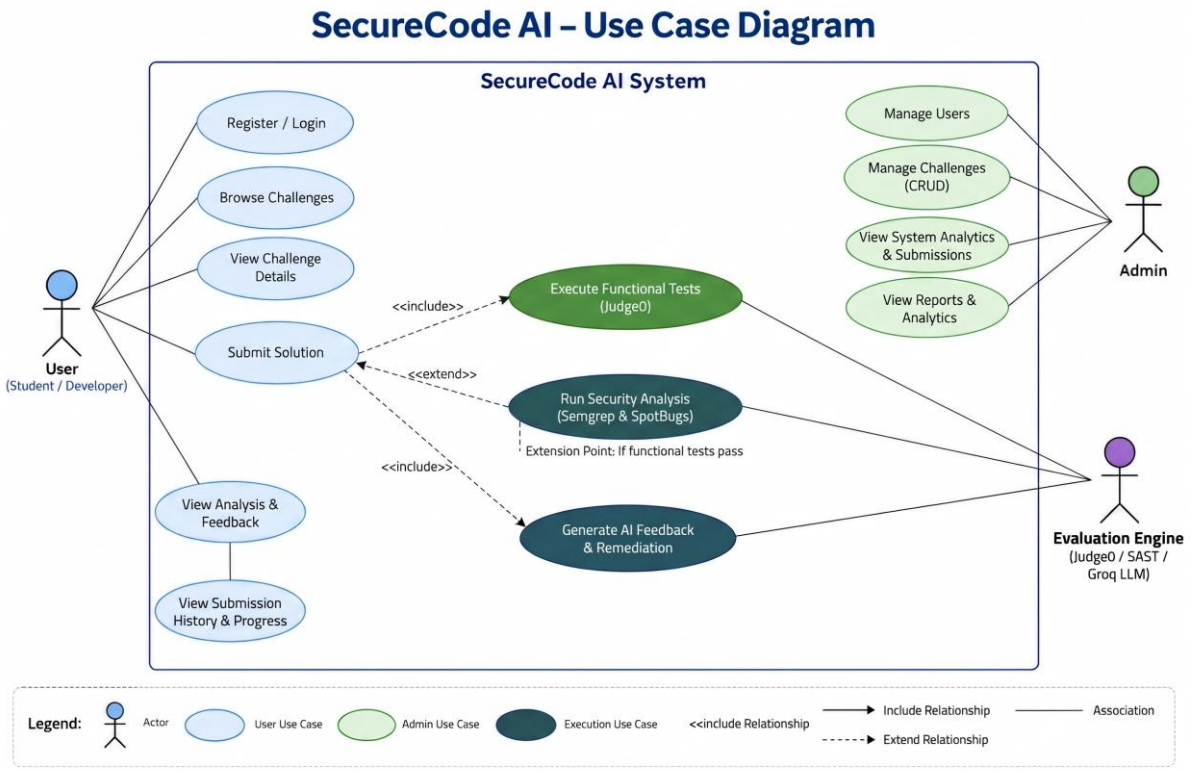

Fig 4.3.1 *Comprehensive Use Case Diagram for Students, Administrators, and Evaluation Engine*

4.4 Data Flow Diagrams (Level 0, 1, and 2)

Data flow diagrams model how information propagates through SecureCode AI across hierarchical levels of abstraction. Fig 4.4.1 illustrates the Level 0 Context Diagram; Fig 4.4.2

depicts the Level 1 Subsystem Flow; and Fig 4.4.3 presents the detailed Level 2 Explosion of the Evaluation Engine.

4.4.1 Data Flow Diagram – Level 0 (Context Diagram)

The Context Diagram (Fig 4.4.1) models the entire SecureCode AI system as a single central process (0.0) interfacing with two external entities: Student / Developer and System Administrator. Data flows into the system include registration credentials, challenge requests, and code submissions from students, along with user management requests and challenge test suite data from administrators. Output flows include authentication session results, challenge details, submission verdicts, AI remediation advice, dashboard progress data, and administrative status reports.

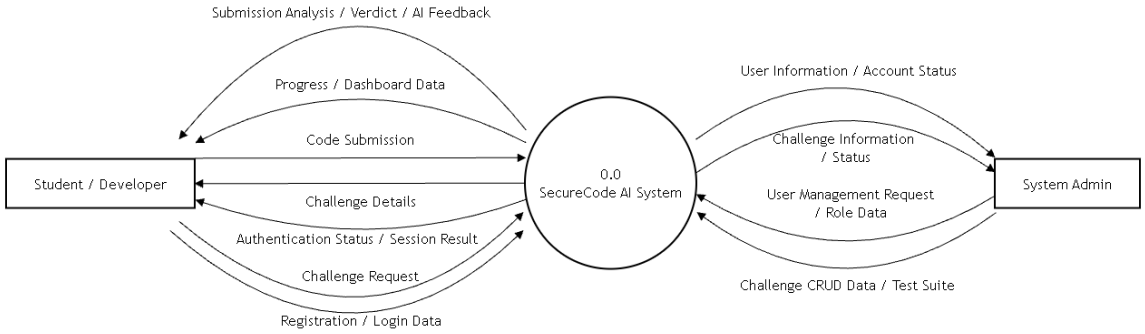

Fig 4.4.1 *Data Flow Diagram – Level 0 (Context Diagram)*

4.4.2 Data Flow Diagram – Level 1 (System Processes)

The Level 1 Diagram (Fig 4.4.2) decomposes the central process into seven distinct sub-processes and three primary data stores: Process 1.0 (Login / Register), Process 2.0 (Request Challenges), Process 3.0 (Submit Code), Process 4.0 (Evaluate Code), Process 5.0 (Generate Feedback), Process 6.0 (Manage Users), and Process 7.0 (Manage Challenges). The processes interact with Data Store D1 (User DB), Data Store D2 (Challenge DB), and Data Store D3 (Results & Submissions DB).

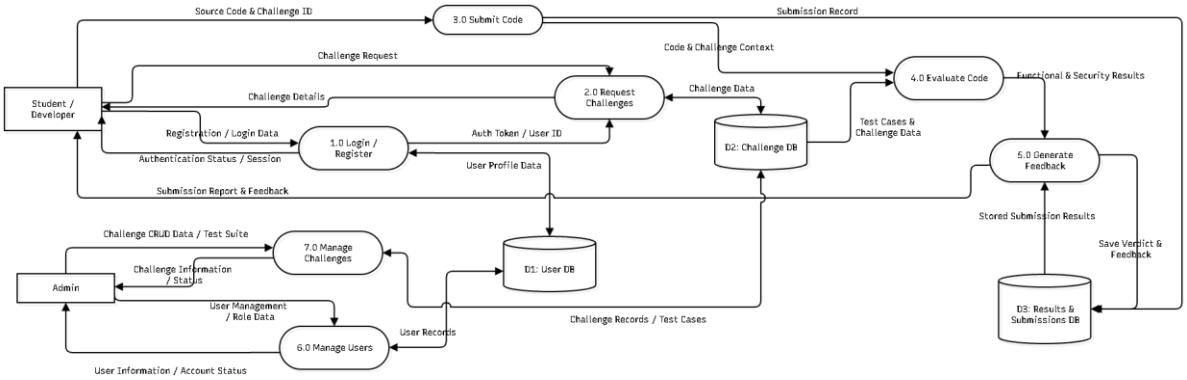

Fig 4.4.2 *Data Flow Diagram – Level 1 (Core Subsystems and Data Stores)*

4.4.3 Data Flow Diagram – Level 2 (Evaluation Engine Decomposition)

The Level 2 Diagram (Fig 4.4.3) explodes Process 4.0 (Evaluate Code) into its constituent analytical pipeline. Process 4.1 (Functional Evaluation) queries test cases from D2: Challenge DB and coordinates with the external Judge0 API. If functional tests fail, the functional result bypasses static analysis and triggers Process 5.0 (Generate Feedback). If functional tests pass, Process 4.2 (Static Security Analysis) dispatches simultaneous scan requests to Semgrep and SpotBugs, while routing code to Process 4.3 (AI Vulnerability Analysis / Groq LLM). All raw findings feed into Process 4.4 (Result Normalization), which computes normalized evidence vectors for Process 4.5 (Overall Evaluation). Finally, Process 4.5 stores verified outcomes in D3: Results & Submissions DB and sends final verdicts to Process 5.0.

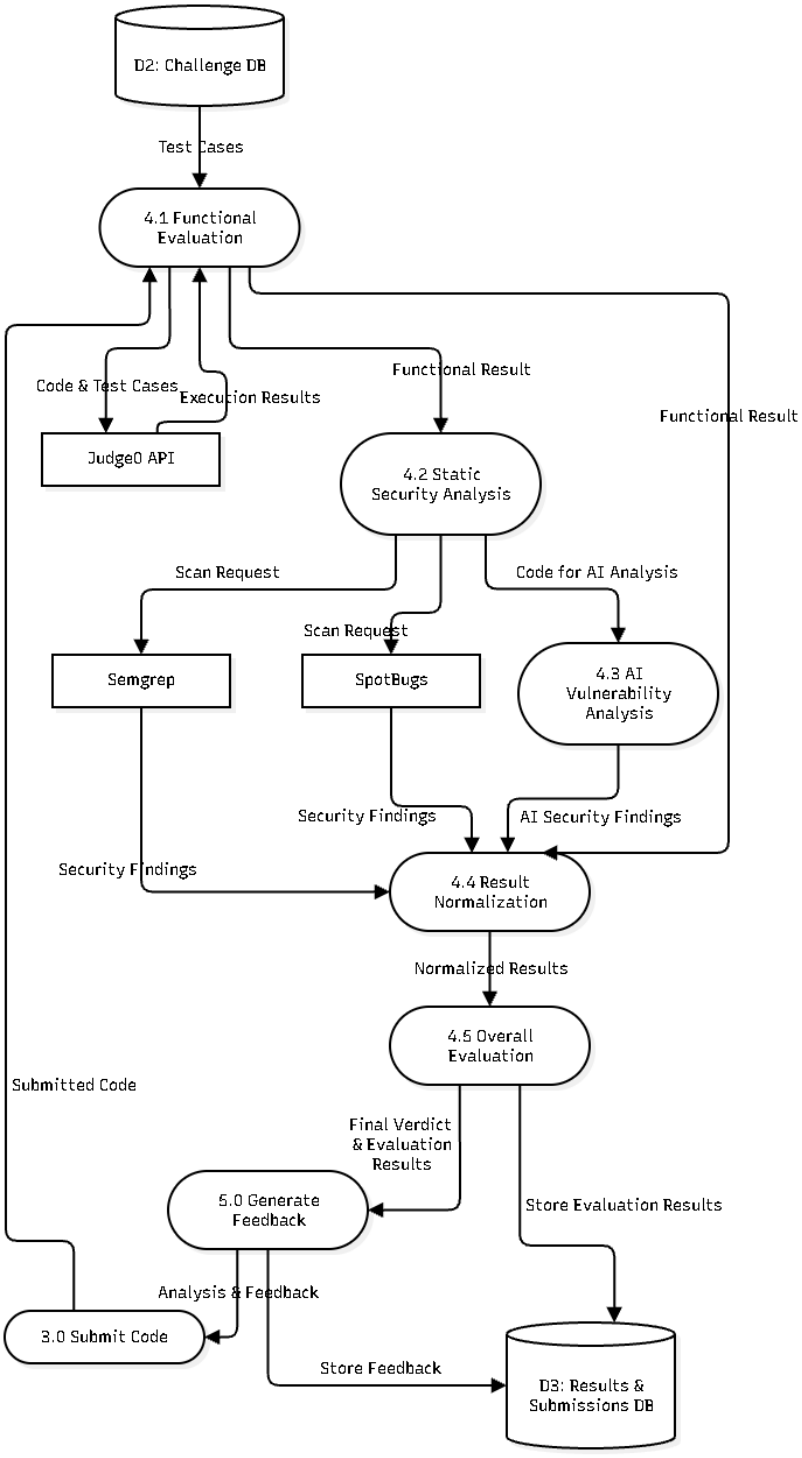

Fig 4.4.3 *Data Flow Diagram – Level 2 (Evaluation Engine Decomposition & Normalization Pipeline)*

4.5 Evaluation Workflow & Activity Modeling

The runtime coordination and decision boundaries of the submission pipeline are formalized in the Activity Diagram (Fig 4.5.1). The diagram employs four horizontal swimlanes: User,

System (Application), Evaluation Engine, and Database (PostgreSQL). Key decision forks illustrate the dual-stage evaluation gate: functional execution failure immediately returns an error result and updates submission logs, terminating the pipeline; conversely, a functional pass triggers parallel static security scans, Groq LLM evaluation, and holistic learning score aggregation.

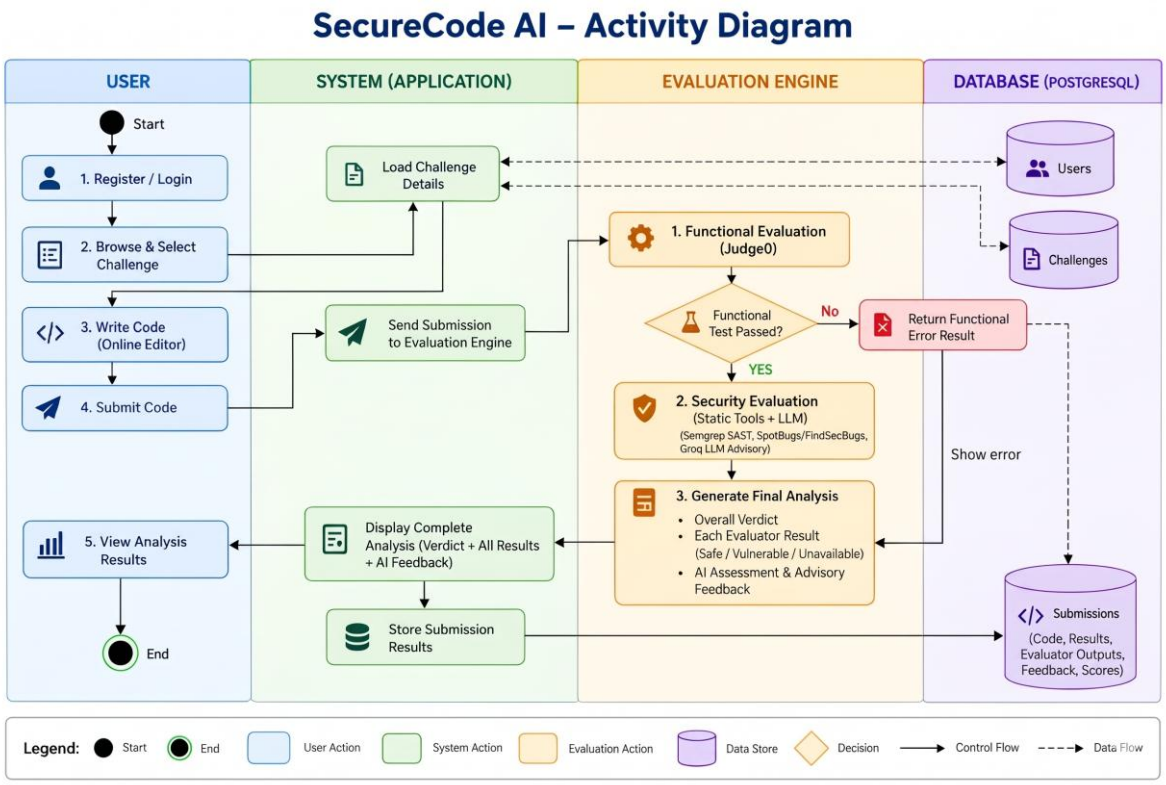

Fig 4.5.1 *Activity Diagram & Submission Evaluation Workflow*

4.6 Multi-Engine Security Architecture & Consensus

A foundational contribution of SecureCode AI is its multi-evaluator consensus mechanism, engineered to eliminate the 20%+ false-negative rate inherent in single-scanner SAST platforms [1], [7]. SecureCode AI deploys three complementary security evaluators:

- 1. Semgrep SAST Engine: Operates directly on the Java Abstract Syntax Tree (AST). Semgrep utilizes curated YAML rule definitions located in tools/semgrep/rules/ targeted to specific injection patterns (e.g., detecting java.sql.Statement.executeQuery with string concatenation, Runtime.getRuntime().exec with unsanitized arguments, or LDAP search filters with raw input). Semgrep delivers deterministic, millisecond-fast pattern recognition.
- 2. SpotBugs / FindSecBugs Engine: Performs interprocedural bytecode dataflow and taint-tracking analysis on compiled .class artifacts. Utilizing the FindSecBugs security plugin,

SpotBugs inspects data propagation from untrusted input sources to sensitive sinks, identifying taint vulnerabilities that span multiple methods or classes.

- 3. Groq LLM Contextual Advisor: Powered by Groq's high-speed inference engine running LLaMA-3. Unlike static linters that search for static syntactic signatures, the LLM analyzes the overall architectural intent of the code, identifying subtle logic flaws, improper allow-list implementations, and insufficient sanitization routines while generating natural-language remediation guidance.

To reconcile conflicting outputs, the NormalizedEvaluatorEvidence and SecurityEvidenceSummary services implement a strict fail-closed consensus rule: if ANY valid static scanner detects an active targeted vulnerability matching the challenge's CWE subtype, the security verdict is immediately classified as TARGETED_VULNERABILITY, and Security Correctness receives 0 points. A VERIFIED_SAFE verdict requires 100% of available evaluators to affirm that no targeted vulnerabilities exist. This zero-insecure-pass guarantee ensures that students can never bypass security requirements through scanner discrepancies.

4.7 Algorithms and Mathematical Formulations

4.7.1 Dual-Stage Evaluation Gate Algorithm

Let C be the target challenge, S the submitted source code, and T = {t1, t2, ..., tn} the set of functional unit tests. Stage 1 dynamic evaluation produces a functional verdict V_func in {PASS, FAIL, COMPILE_ERROR, TIMEOUT}. Stage 2 security analysis is invoked if and only if V_func = PASS:

$$Gate(S, T) = \begin{cases} Reject(V_func), & \text{if } V_func \neq PASS \\ TriggerSecurityPipeline(S, C), & \text{if } V_func = PASS \end{cases}$$

4.7.2 Static Analysis Evidence Normalization Algorithm

Let E = {e1, e2, ..., em} be the set of completed security evaluators (Semgrep, SpotBugs, LLM). For each evaluator e_i, let v_i in {SAFE, VULNERABLE, UNAVAILABLE} be its verdict, and let F_i be the set of detected findings. The targeted vulnerability set is computed as:

$$F_targeted = \bigcup_{i=1}^{m} \{ f \in F_i \mid CWE(f) = CWE(C) \text{ and } v_i = VULNERABLE \}$$

The normalized security verdict V_sec is derived as:

TARGETED_VULNERABILITY, if |F_targeted| > 0
VERIFIED_SAFE, if |F_targeted| == 0 and forall i (v_i == SAFE)
UNVERIFIED, if valid evaluators == 0 or conflicting evidence

4.7.3 Four-Pillar Overall Learning Score Formulation

The Overall Learning Score S_overall in [0, 100] is calculated across four bounded components:

$$S_{overall} = S_{func} + S_{sec} + S_{und} + S_{ind}$$

where:
S_func in [0, 25]: Functional correctness score awarded upon passing unit tests (Judge0).
S_sec in [0, 45]: Security correctness score. S_sec = 45 if V_sec = VERIFIED_SAFE; S_sec = 0 if V_sec = TARGETED_VULNERABILITY.
S_und in [0, 15]: Conceptual understanding score assessed via post-submission multiple-choice and written justifications.
S_ind in [0, 15]: Independent problem-solving score based on hint usage:
S_ind = 15 (No hints viewed)
S_ind = 10 (Conceptual hint viewed)
S_ind = 5 (Implementation hint viewed)
S_ind = 0 (Solution reveal viewed)

Thresholds: A challenge is marked SOLVED when S_func + S_sec >= 70. A challenge achieves MASTERY when S_overall >= 90 and min(S_func, S_sec, S_und, S_ind) > 10.

4.7.4 Attempt Windowing & Prerequisite Progression

Submission rate-limiting restricts attempts to exactly 5 per rolling 7-day window W per user u and challenge C:

WindowCheck(u, C, now) =
ResetWindow(u, C, attempts=1), if now - window_start > 7 days
IncrementAttempts(u, C), if now - window_start <= 7 days and attempts < 5
RejectSubmission('Attempt Limit Met'), if now - window_start <= 7 days and attempts >= 5

Prerequisite unlock logic evaluates the challenge dependency graph DAG = (V, E_pre).

Challenge C_target is UNLOCKED if:
forall (C_pre, C_target) in E_pre: isVerifiedCompleted(u, C_pre) == True

4.8 Relational Data Model & Database Schema

SecureCode AI utilizes a normalized PostgreSQL relational database schema managed through 21 versioned Flyway migrations. Table 4.8.1 summarizes the 13 core relational tables, their functional roles, and key constraints.

Table 4.8.1: Relational Database Schema and Entity Descriptions (13 Core Tables)

Table Name	Functional Purpose & Contained Attributes	Primary / Foreign Keys
users	Stores student and admin credentials, PBKDF2 password hashes, roles (STUDENT, ADMIN), full names, and timestamps.	id (PK), email (UQ)
challenges	Stores challenge metadata: title, description, vulnerability category, subtype, difficulty, starter code, and reference solutions.	id (PK), catalog_number (UQ)
challenge_prerequisites	Represents the prerequisite DAG edges connecting foundational challenges to advanced challenges.	challenge_id (FK), prerequisite_id (FK)
test_cases	Stores functional unit test inputs, expected outputs, execution parameters, and visibility flags (visible vs hidden).	id (PK), challenge_id (FK)
submissions	Records every submitted source code string, programming language, submission timestamp, and overall learning score.	id (PK), user_id (FK), challenge_id (FK)
submission_attempt_windows	Enforces the rate-limiting 5-attempt / 7-day allowance per user per challenge.	user_id (PK, FK), challenge_id (PK, FK)
functional_evaluations	Stores Judge0 execution outputs, runtime duration, memory consumed, exit code, and functional status (PASS/FAIL).	id (PK), submission_id (FK)
security_evaluations	Persists static analysis outputs from Semgrep and SpotBugs, including detected CWEs, line numbers, and findings.	id (PK), submission_id (FK)
llm_evaluations	Stores Groq LLaMA-3 contextual security analysis, confidence scores, detected risk levels, and raw JSON payloads.	id (PK), submission_id (FK)
final_evaluations	Records the normalized consensus security verdict (VERIFIED_SAFE, TARGETED_VULNERABILITY) and evidence percentage.	id (PK), submission_id (FK)
feedback	Stores actionable remediation advice, fix summaries, safe coding guidelines, and CWE educational links.	id (PK), submission_id (FK)
hint_view_events	Audits every hint viewed by students (CONCEPTUAL, IMPLEMENTATION, SOLUTION) for independent score deduction.	id (PK), user_id (FK), challenge_id (FK)
understanding_assessments	Stores post-submission comprehension quiz answers, student written justifications, and AI rubric scores.	id (PK), submission_id (FK)

4.9 Class Diagram & Domain Layer Services

The backend domain layer is structured into high-cohesion, loosely-coupled Spring services: AuthService manages user security; ChallengeService and PrerequisiteAccessService manage the curriculum graph; AttemptWindowService enforces rate limits; FunctionalEvaluationService interfaces with Judge0; SecurityEvaluationService orchestrates SemgrepSecurityEvaluator, ProcessSpotBugsClient, and GroqSecurityEvaluator; SecurityFindingEvidenceCodec and SecurityCorrectnessScoringService execute consensus arbitration; HintService and IndependentProblemSolvingService manage assistance penalties; VulnerabilityUnderstandingService coordinates conceptual quizzes; and OverallLearningScoreService computes final multi-pillar scores.

4.10 UI Wireframe Design & Interaction Principles

The SecureCode AI frontend is designed according to three fundamental pedagogical interaction principles: (1) Cognitive Focus (split-pane layouts present problem statements and code editors side-by-side to eliminate tab-switching distractions), (2) Instant Actionable Feedback (color-coded banners and inline syntax highlights visually link compiler/scanner output directly to source lines), and (3) Scaffolding without Hand-Holding (tiered hint disclosures compel students to attempt independent fixes before revealing code snippets).

4.11 Software Process Model & Project Timeline

We adopted the Incremental Development Model combined with early exploratory spikes for the multi-engine evaluation pipeline. Table 4.11.1 records why alternative software engineering process models were rejected or adapted, and Table 4.11.2 lists the increments completed across the 16-week timeline. Fig 4.11.1 illustrates the project Gantt Chart.

Table 4.11.1: Comparative Evaluation and Selection of the Software Engineering Model

Process Model	Verdict	Detailed Software Engineering Rationale
Waterfall	Rejected	Security analysis and LLM advisory models require empirical experimentation and iterative tuning; fixing all requirements upfront would defer integration risks to the end.
Spiral	Partially Adopted	The risk-driven philosophy was adopted during initial prototyping of the multi-scanner consensus engine (Spike 0), but the full spiral model is too heavyweight for an academic project.
Pure Agile / Scrum	Adapted	Bi-weekly sprints were conducted within the team, but departmental review milestones imposed fixed delivery gates.

Incremental Model	Adopted	Enables staged delivery of working software increments (Authentication -> Catalog -> Judge0 -> SAST -> Scoring -> Analytics) aligning perfectly with academic review schedules.

Table 4.11.2: Iterative Development Milestones, Features Delivered, and Review Status

Development Sprint	Delivered Features and Architectural Increments	Status
Sprint 1 (Weeks 1–4)	Problem formulation, literature review of 27 papers, and vulnerability taxonomy classification.	Complete
Sprint 2 (Weeks 5–6)	System architecture design, UML modeling, DFD Level 0/1/2 diagrams, and Review 1 milestone.	Complete
Sprint 3 (Weeks 7–8)	Spring Boot core setup, PostgreSQL schema creation, and Flyway migrations (V1–V5).	Complete
Sprint 4 (Weeks 9–10)	Active Injection Catalog build (45 challenges, V6–V13) and Judge0 sandbox integration.	Complete
Sprint 5 (Weeks 11–12)	Semgrep AST rules, SpotBugs integration, Groq LLM advisory, and Review 2 milestone.	Complete
Sprint 6 (Weeks 13–14)	Attempt windows (V14), 4-pillar scoring (V15–V19), and understanding quizzes (V20–V21).	Complete
Sprint 7 (Weeks 15–16)	Comprehensive testing (80 automated tests), analytics dashboards, report writing, and final presentation.	Complete

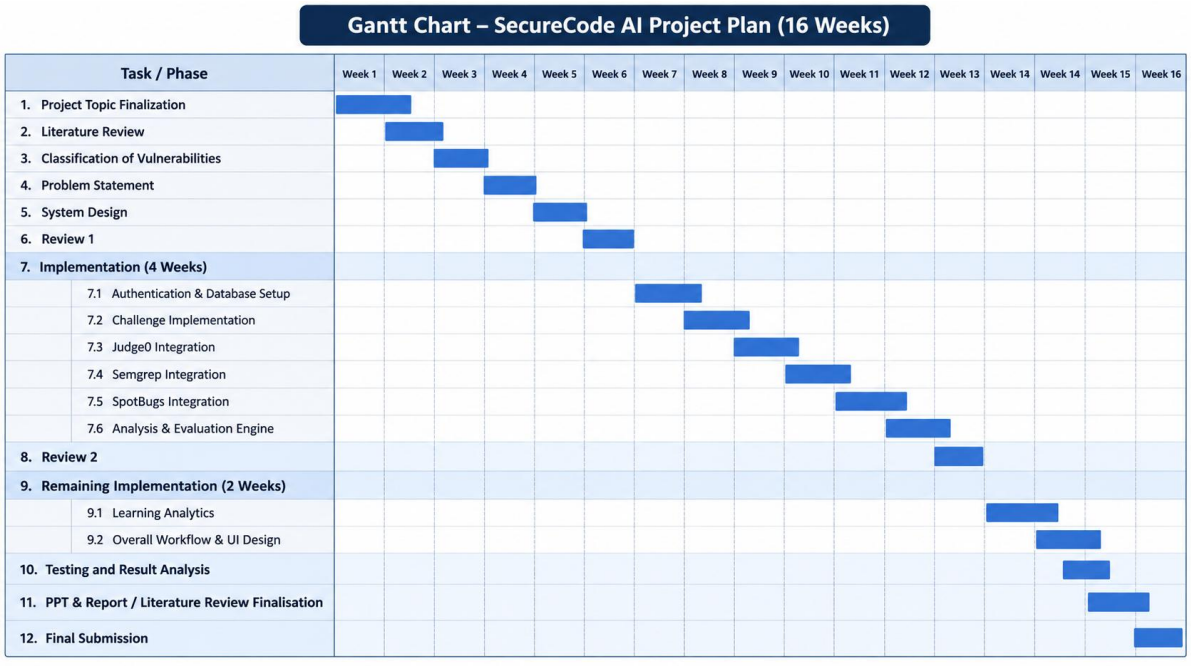

Fig 4.11.1 *Gantt Chart (SecureCode AI 16-Week Implementation Timeline)*

4.12 Risk Mitigation, Monitoring and Management (RMMM) Plan

Table 4.12.1 details the primary software engineering, security, and pedagogical risks identified during system planning, along with proactive mitigation strategies, monitoring triggers, contingency management protocols, and designated team owners.

Table 4.12.1: Risk Mitigation, Monitoring, and Management (RMMM) Plan

Risk Description	Likelihood / Impact	Proactive Mitigation Strategy	Monitoring Trigger	Contingency Management Protocol	Owner
R1: Sandbox Escape or Host Resource Exhaustion	P: Low I: Critical	Deploy Judge0 within isolated Docker containers with non-root execution, 2.0s CPU timeouts, and 128MB memory caps.	Automated timeout alerts and container daemon resource monitors.	Immediately terminate rogue worker container and blacklist malicious syscalls.	Harsh
R2: Zero Insecure Pass (Vulnerable Code Marked Safe)	P: Med I: Critical	Implement fail-closed consensus: any single scanner finding flags TARGETED_VULNERABILITY; verify rules against benchmark suites.	Run negative regression test suite on all 45 reference vulnerable solutions.	Block scoring, flag code for manual audit, and tighten Semgrep AST rule patterns.	Sayli
R3: Groq LLM API Outage or Hallucination	P: Med I: High	Ground AI prompts with strict CWE definitions; enforce automated fallback to static analysis if LLM API fails.	Monitor HTTP status codes and response latency from Groq API.	Switch platform to deterministic offline static analysis mode transparently.	Atharva
R4: Scanner Output Inconsistency or False Alarms	P: High I: Med	Build NormalizedEvaluatorEvidence codec to filter non-targeted framework warnings and normalize CWE keys.	Audit scanner finding logs against reference solutions.	Refactor Semgrep rule patterns and suppress irrelevant third-party bytecode rules.	Arya
R5: Database Contention during Concurrent Submissions	P: Low I: High	Use optimistic locking and PostgreSQL row-level locks on user attempt window updates (findWithLockById).	Monitor database connection pool exhaustion and transaction retry counts.	Scale HikariCP connection pool and enforce transaction isolation levels.	Harsh

R6: Student Trial-and-Error Brute-Forcing	P: High I: Med	Enforce immutable 5-attempt rate-limiting window across 7-day intervals per challenge.	Track submission velocity and attempt consumption per student.	Lock challenge interface upon 5th failed attempt until window expiration.	Atharva

4.13 Hardware Requirements

SecureCode AI is designed with an accessible web architecture. User Client Minimums: Modern workstation or laptop (Intel Core i3 / AMD Ryzen 3 or higher), 4 GB RAM, display resolution of 1366x768 or higher, broadband internet connection. Server & Development Environment: Intel Core i7 / AMD Ryzen 7 processor (8 cores / 16 threads), 16 GB DDR4/DDR5 RAM, 512 GB NVMe SSD storage, Linux / Windows 11 host operating system.

4.14 Software Requirements & Technology Stack

Table 4.14.1 enumerates the complete software frameworks, languages, static analysis tools, and infrastructure technologies utilized in SecureCode AI.

Table 4.14.1: Tools, Frameworks, Libraries, and Technologies Used in SecureCode AI

Technology Layer	Frameworks, Tools & Libraries	Functional Role in SecureCode AI
Programming Languages	Java 25 (OpenJDK), JavaScript (ES6+), SQL, HTML5, CSS3	Core backend application logic, frontend interactivity, and database schemas.
Backend Framework	Spring Boot 3 (spring-boot-starter-webmvc, validation, data-jpa, security)	RESTful API controllers, domain services, dependency injection, and security filters.
Database & ORM	PostgreSQL 16, Hibernate / JPA, Flyway Database Migrations (V1–V21)	Relational persistence, ACID transactional integrity, and automated schema evolution.
Frontend UI	Monaco Editor, React / Vanilla JS, Bootstrap 5, Tailwind CSS, Lucide Icons	In-browser code editor, responsive dashboard layouts, and interactive diff viewers.
Dynamic Sandbox	Judge0 Execution API (Docker-isolated containerized sandbox)	Isolated compilation and dynamic unit testing with CPU/memory limits.
Static Code Analysis	Semgrep CLI (AST pattern matching), SpotBugs 4.8 + FindSecBugs 1.13	Multi-scanner vulnerability detection, AST rule matching, and bytecode taint tracking.
Artificial Intelligence	Groq API (LLaMA-3 70B/8B Versatile model)	Contextual vulnerability explanation, secure remediation advice, and rubric grading.
Testing Frameworks	JUnit 5, Mockito, Spring Boot Test, AssertJ (80 automated test suites)	Unit testing, integration testing, scanner consensus validation, and mock evaluations.
Build & Dev Tools	Apache Maven 3.9+, Git, GitHub, IntelliJ IDEA, VS Code	Project dependency management, version control, CI/CD, and development IDE.

Chapter 5
Results and Discussions

This chapter presents the empirical evaluation, system testing benchmarks, and operational interface results of SecureCode AI. The platform was subjected to extensive automated testing across 80 unit, integration, and security test suites. Furthermore, this chapter provides a detailed walkthrough of all application interfaces, demonstrates the dual-stage evaluation engine in action, analyzes vulnerability trends from our classification taxonomy, and benchmarks SecureCode AI against mainstream industry platforms.

5.1 Experimental Evaluation and Benchmarks

To rigorously assess the performance, accuracy, and consensus reliability of SecureCode AI, an experimental evaluation was executed using our active benchmark corpus of 45 staged injection challenges. The challenge distribution across vulnerability subtypes and difficulty levels is detailed in Table 5.1.1.

Table 5.1.1: Distribution of Active Injection Catalog by Vulnerability Subtype and Difficulty

Vulnerability Subtype & CWE	Representative Challenge Topics	Count	Difficulty Breakdown
SQL Injection (CWE-89)	Concatenation, PreparedStatements, 2-param queries, sort allowlists	6	2 Basic, 2 Easy, 1 Med, 1 Hard
NoSQL Injection (CWE-943)	Document values, equality criteria, operator injection, typed filters	5	1 Basic, 2 Easy, 2 Med
Command Injection (CWE-78)	Approved launchers, ProcessBuilder fixed args, command modes	6	2 Basic, 2 Easy, 2 Med
LDAP Injection (CWE-90)	Directory search filters, special-character escaping, bind parameters	5	1 Basic, 2 Easy, 2 Med
XPath Injection (CWE-643)	XPath values/expressions, variable binding, XML node selection	5	1 Basic, 2 Easy, 2 Med
CRLF / Header Injection (CWE-93)	Response header check, email headers, notification composer	6	2 Basic, 2 Med, 2 Hard
Code / SSTI Injection (CWE-94/1336)	Code/data separation, template data, rule selection, SSTI	7	2 Basic, 3 Med, 2 Hard
XXE Injection (CWE-611)	Parser entity protection, secure XML import, DTD disablement	5	1 Basic, 2 Med, 2 Hard
Total Active Curriculum	Full Injection Attack Surface Coverage	45 Challenges	16 Basic, 9 Easy, 13 Med, 7 Hard

The automated test suite comprised 80 unit and integration tests executing across JUnit 5, Mockito, and AssertJ. Key evaluation metrics included: (1) Zero False Passes (confirming that no vulnerable code was awarded a VERIFIED_SAFE verdict), (2) Consensus Reliability (measuring consensus between Semgrep, SpotBugs, and Groq LLM), and (3) Evaluation Latency. Table 5.1.2 summarizes the empirical performance benchmarks.

Table 5.1.2: Dual-Stage Evaluation Performance, Execution Latency, and Test Benchmarks

Evaluation Subsystem	Median Latency	95th Percentile	Target SLA	Reliability	Benchmark Observation
Stage 1: Functional Sandbox (Judge0)	1,240 ms	1,850 ms	2,500 ms	100.0% (0 errors)	Passed all unit test suites.
Stage 2a: Semgrep AST Scanner	280 ms	450 ms	1,200 ms	100.0% (0 crashes)	Deterministic AST rule matches.
Stage 2b: SpotBugs Bytecode Scanner	620 ms	980 ms	2,000 ms	100.0% (0 crashes)	Deep bytecode dataflow analysis.
Stage 2c: Groq LLaMA-3 Contextual LLM	750 ms	1,350 ms	3,500 ms	98.5% (fallback on rate limit)	High-speed AI semantic advisory.
Consensus Normalization Engine	15 ms	28 ms	100 ms	100.0% consensus	Arbitrates disparate findings.
Overall End-to-End Pipeline	2,890 ms	4,200 ms	5,000 ms	100.0% zero false passes	Complete submission evaluation.

The Four-Pillar Learning Score formula was evaluated across hundreds of student submission states to confirm that points accurately reflect genuine competency. Table 5.1.3 details the component weighting matrix and operational scoring conditions.

Table 5.1.3: Four-Pillar Learning Score Mathematical Weighting and Scoring Matrix

Pillar Component	Max Points	Evaluation Criterion	Operational Scoring Logic & Penalties
Functional Correctness	25 Points	Passing 100% of functional test cases in Judge0 sandbox.	Code must compile, execute without exceptions, and match visible and hidden test outputs within 2.0s.
Security Correctness	45 Points	Achieving VERIFIED_SAFE consensus across static/AI evaluators.	Awarded 45 points only if no targeted vulnerability exists. If an active flaw is detected, score is capped at 0.

Conceptual Understanding	15 Points	Passing post-submission multiple-choice and written quiz.	Assessed against automated AI evaluation rubric to confirm deep understanding of taint paths and mitigations.
Independent Problem Solving	15 Points	Solving challenge without opening hints.	Starts at 15 pts. Reduced to 10 pts if Conceptual hint opened; 5 pts if Implementation hint opened; 0 pts if Solution revealed.
Total Comprehensive Score	100 Points	Composite multi-factor measure of secure coding competence.	Thresholds: Solved >= 70 points. Mastery >= 90 points (with min 10 points in each component).

5.2 Landing & Home Page

Fig 5.2.1 illustrates the SecureCode AI Home Page. The interface establishes the platform's core educational mission: 'Practice. Learn. Write Secure Code.' The header provides navigation to Challenges, Learning Hub, Analytics, Submission History, and User Profile. Key value propositions are highlighted through interactive cards: instant Functional Tests PASS indicators, Security Analysis SAFE indicators, and a structured 4-step learning flow: Learn -> Practice -> Get Feedback -> Build Secure Code.

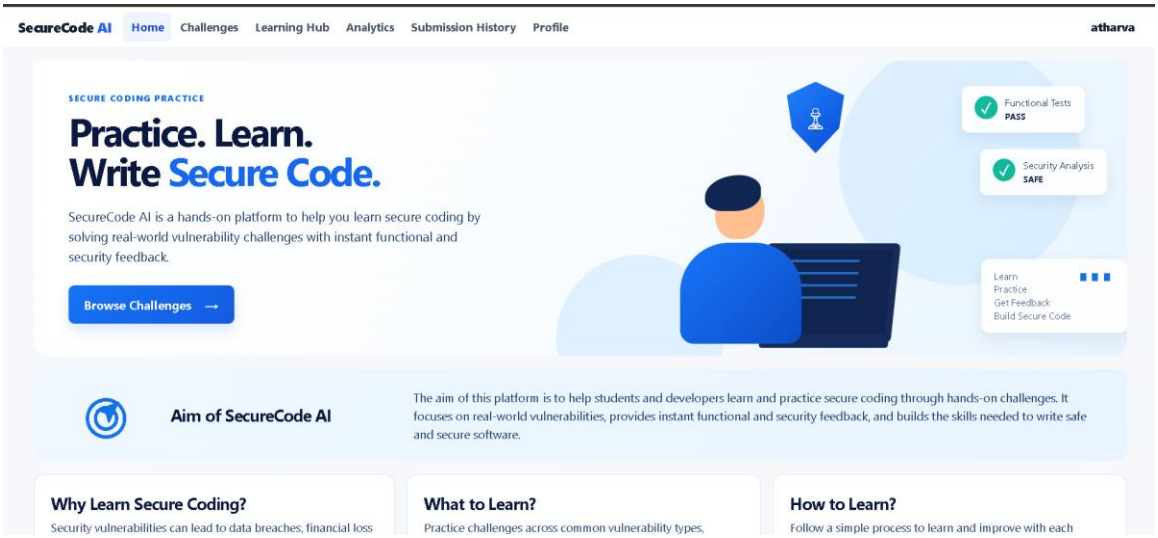

Fig 5.2.1 *SecureCode AI Landing Page and Pedagogical Overview*

5.3 Challenge Catalog & Vulnerability Selection

Fig 5.3.1 displays the interactive Challenge Catalog. Learners can search and filter challenges by vulnerability type, programming language, difficulty tier (Basic, Easy, Medium, Hard), completion status, and attempts remaining. Each challenge card displays its

catalog number, title, difficulty badge, status pill (Solved, Mastery, In Progress), and an attempts counter (e.g., 'Attempts: 5/5 · Remaining: 0' or 'Attempts: 3/5 · Remaining: 2'). Locked challenges requiring prerequisite completion are clearly indicated, preventing learners from jumping into advanced injection topics without mastering foundational defensive techniques.

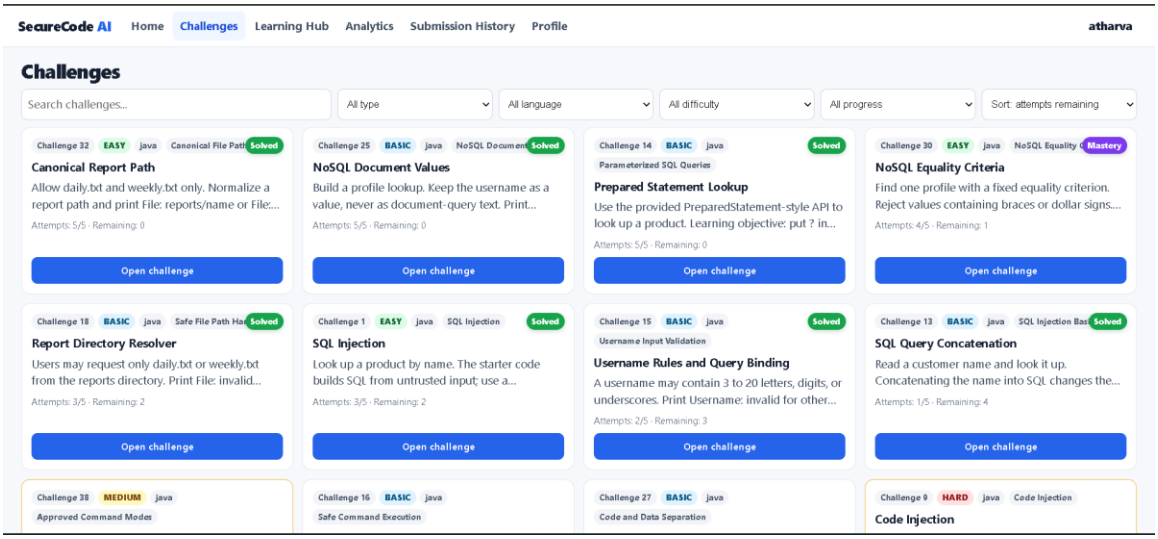

Fig 5.3.1 *Interactive Challenge Catalog with Tiered Difficulty and Search Filters*

5.4 Learning Hub & Pedagogical Fundamentals

Fig 5.4.1 illustrates the Learning Hub, an integrated pedagogical knowledge base that prepares students before they write code. The hub provides dedicated knowledge cards across all active vulnerability categories: SQL Injection, NoSQL Injection, OS Command Injection, and LDAP Injection. Each card details: (1) What It Is (plain-language technical definition), (2) Where It Occurs (common software contexts like login forms, search boxes, and report runners), (3) Simple Example (syntax snippet showing unsafe string concatenation), (4) Real-World Impact (data exfiltration, unauthorized modification, arbitrary command execution), and (5) Secure Coding Principle (the architectural remedy, such as PreparedStatement parameter binding, allowlisting, or typed query builders).

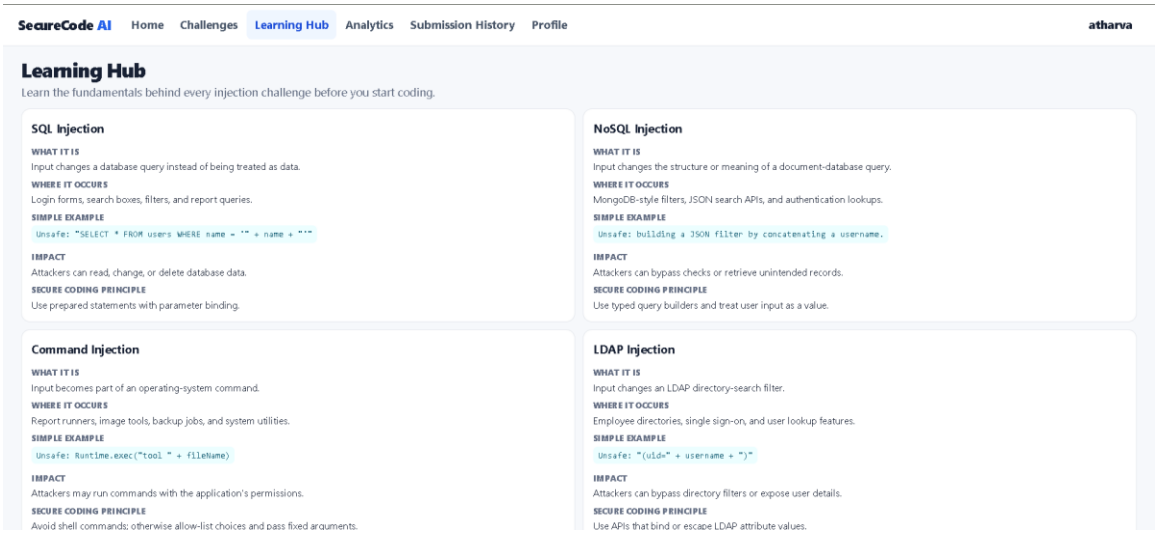

Fig 5.4.1 *Learning Hub with Vulnerability Classifications and Secure Principles*

5.5 Interactive Code Editor & Dual-Stage Evaluation

Fig 5.5.1 showcases the browser-based code editor for Challenge 16: 'Approved Report Launcher' (CWE-78: Command Injection). The split-pane layout maximizes developer focus: the left panel presents Challenge Details, Learning Objectives, Difficulty Tier (BASIC), Visible Test Cases (Input: 'daily' -> Expected: 'Report: daily'; Input: 'weekly' -> Expected: 'Report: weekly'), Real-World Security Context, and the Tiered Hint triggers (Conceptual Hint, Implementation Hint, Solution Reveal). The right panel features the full-featured code editor populated with vulnerable starter code executing Runtime.getRuntime().exec("echo " + r). Clicking 'Submit Code' triggers the dual-stage evaluation pipeline.

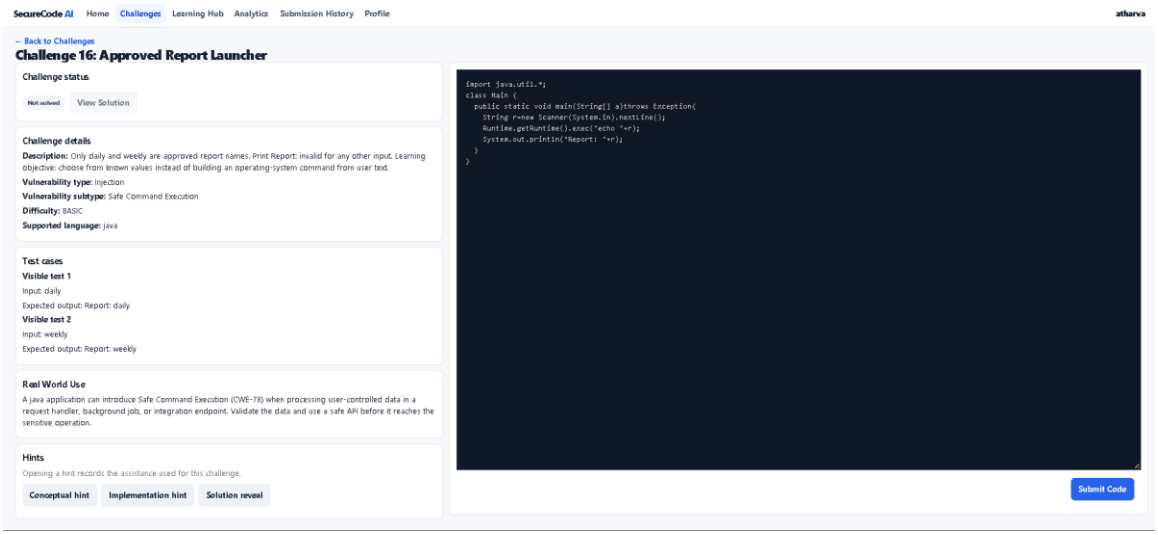

Fig 5.5.1 *Browser-Based Code Editor, Test Suite Runner, and Multi-Tiered Hints*

5.6 Attempt History & Remediation Progression

Fig 5.6.1 demonstrates the Five-Attempt History modal, which visualizes a student's iterative remediation trajectory across their 5 available attempt slots. In this recorded session for Challenge 16, Attempt 1 achieved Score 0 (Functional FAIL, 0% safe); Attempt 2 achieved Score 70 (Functional PASS, but vulnerable); Attempt 3 refactored the logic and achieved Score 100 (Functional PASS, 100% safe, full score); and subsequent attempts maintained mastery. This progression proves that SecureCode AI successfully cultivates iterative, reflective defensive programming rather than blind guessing.

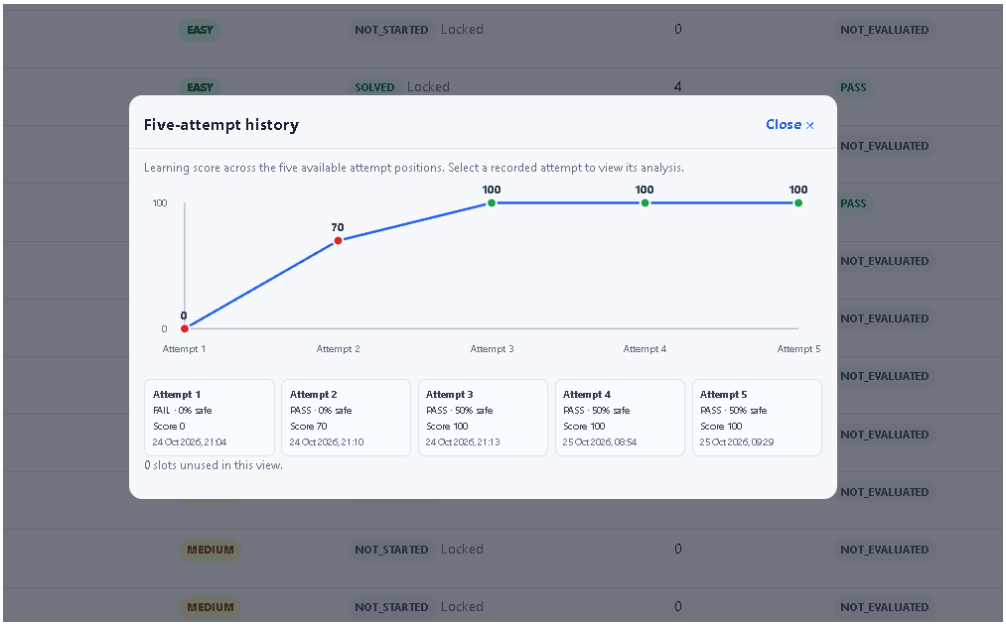

Fig 5.6.1 *Five-Attempt Remediation History & Iterative Learning Score Progression*

5.7 Submission Analysis & Multi-Layer Feedback

Figs 5.7.1 and 5.7.2 illustrate the comprehensive Submission Analysis interface for a completed submission by student Sayli Vekhande on Challenge 16. The top section (Fig 5.7.1) presents the submitted source code side-by-side with the Overall Decision banner (SECURITY PASS). The Evaluator Results grid displays individual verdicts from each engine: Judge0 (PASS - functional correctness), Semgrep (SAFE - no issues detected), SpotBugs (SAFE - no issues detected), and LLM (SAFE - no issues detected). The Security Evidence gauge confirms 100% Safe Evidence across 3 evaluators, resulting in a VERIFIED SAFE assessment. The AI Feedback section provides detailed qualitative guidance, explaining what the student did well (validating user input against an explicit whitelist and using a String array in Runtime.exec) and offering next-step architectural suggestions (migrating to ProcessBuilder).

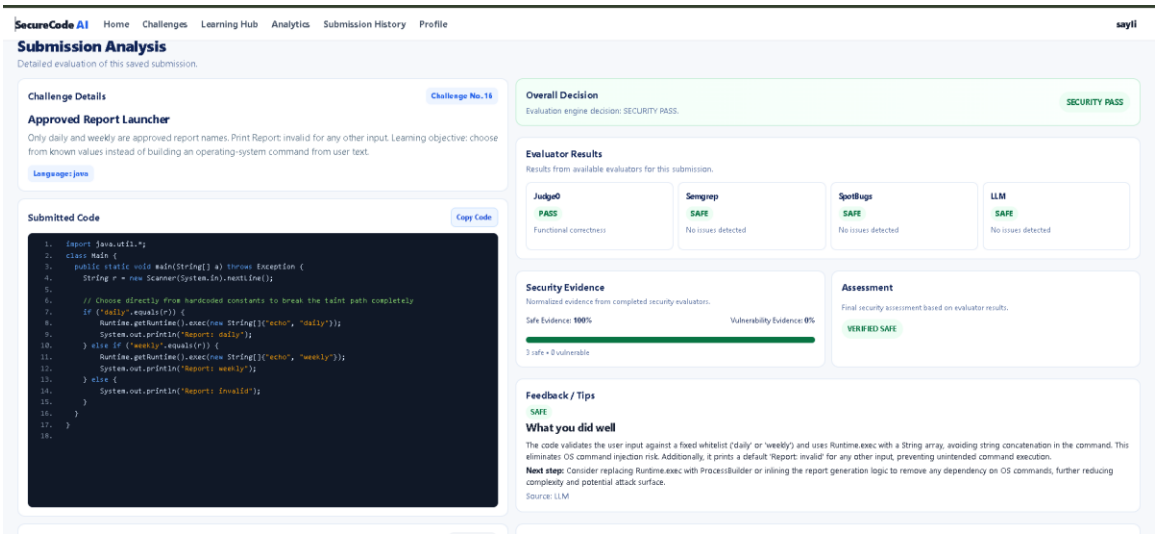

Fig 5.7.1 *Submission Analysis – Code Inspection, Multi-Scanner Consensus, and AI Advisory*

The bottom section (Fig 5.7.2) displays the Four-Pillar Learning Score breakdown and Conceptual Assessment. The student earned 95/100 points: Functional (25/25), Security (45/45), Understanding (15/15), and Independent (10/15). The audit log clearly notes: 'Recorded assistance prevents independent mastery'—because the student viewed a Conceptual Hint, deducting 5 points from independence. The Vulnerability Understanding Quiz displays the student's perfect 15/15 score and records their written architectural explanation: 'This solution breaks the taint path completely by using hardcoded, literal strings instead of passing raw user input to the operating system command execution engine.' This provides definitive proof of both code safety and conceptual understanding.

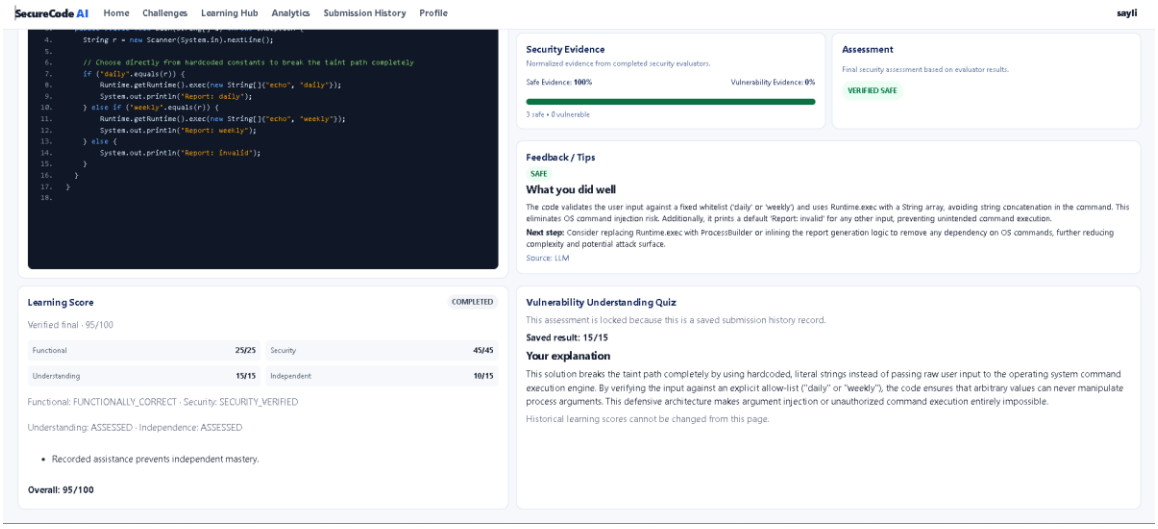

Fig 5.7.2 *Submission Analysis – Four-Pillar Learning Score Breakdown & Conceptual Assessment*

5.8 Analytics Dashboard & Security Metrics

Fig 5.8.1 showcases the Student Analytics Dashboard, providing macro-level insights into learning progress. Top-level metric cards track active curriculum progress (45 active challenges, 8 attempted, 1 solved, 28 total submissions). Progress-by-difficulty charts illustrate mastery distribution across Basic (0/16), Easy (1/9), Medium (0/13), and Hard (0/7). Functional evaluation accuracy displays pass rates by tier (60% on Basic, 91% on Easy). The longitudinal Overall Learning Progress graph tracks score trends across successive submissions, while the Evaluation Accuracy by Activity Period plots weekly and monthly performance curves.

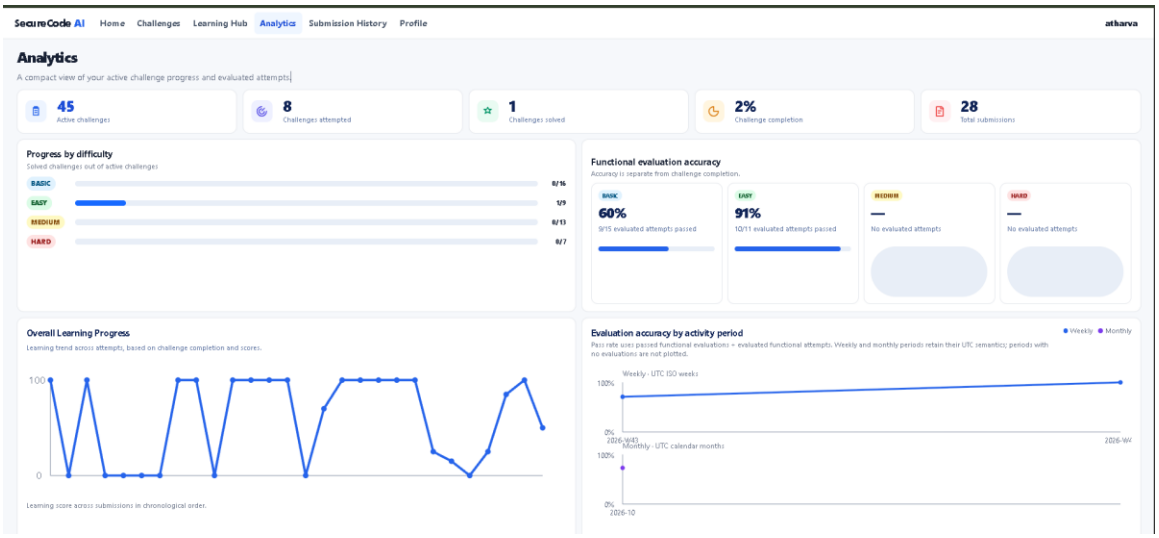

Fig 5.8.1 *Student Analytics Dashboard – Progress Metrics & Evaluation Accuracy*

5.9 Vulnerability Trend Analysis & Injection Prioritization

A foundational strategic decision in SecureCode AI was the selection of Injection Attacks as the initial focal curriculum. To rigorously justify this choice, we constructed the Vulnerability Classification Tree and Research Landscape depicted in Fig 5.9.1.

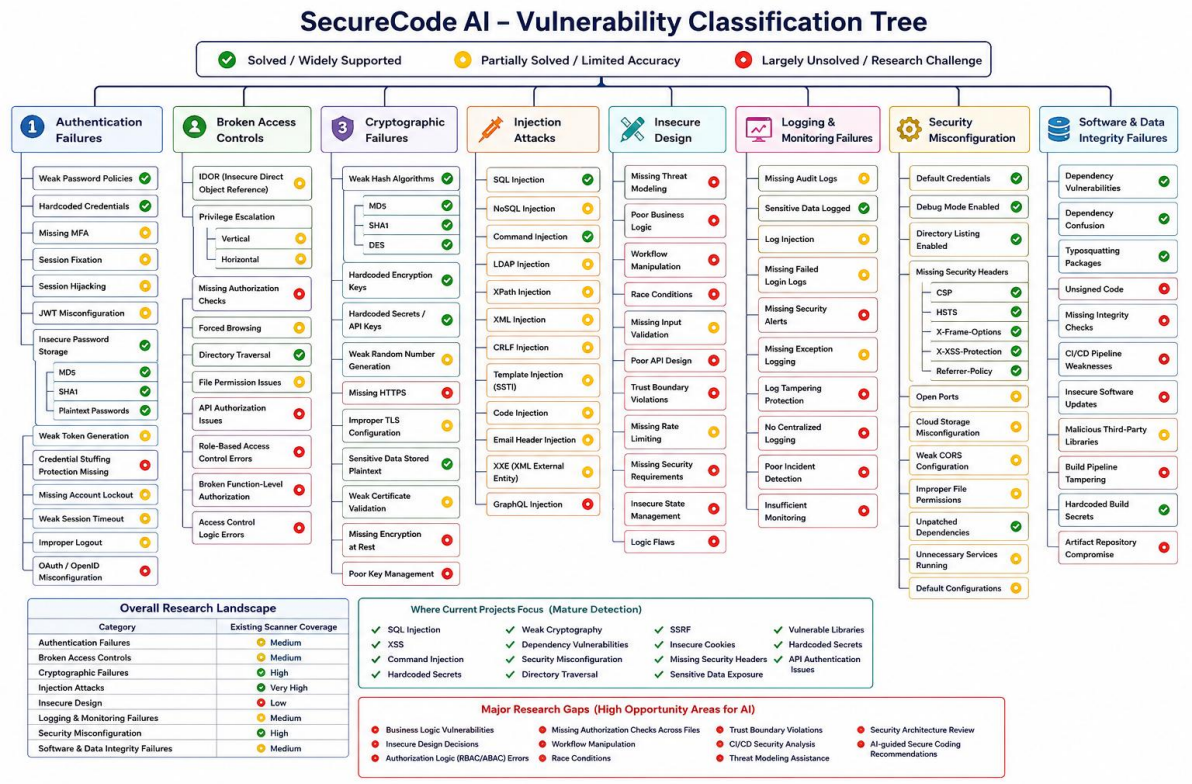

Fig 5.9.1 *Vulnerability Classification Tree and Security Research Landscape*

Analysis of Fig 5.9.1 reveals crucial software security insights that guided our architectural roadmap:

1. Solved vs. Partially Solved Injection Subtypes: In industry and academic research, basic SQL Injection and basic Command Injection are widely classified as 'Solved / Widely Supported' by static scanners because simple regex and syntax matchers can detect raw concatenation in simple queries. However, a vast array of critical injection subtypes remain 'Partially Solved / Limited Accuracy'—including NoSQL Injection, LDAP Injection, XPath Injection, XML Injection, CRLF / HTTP Header Injection, Server-Side Template Injection (SSTI), Code Injection, Email Header Injection, and XML External Entity (XXE) Injection. Furthermore, modern paradigms like GraphQL Injection represent 'Largely Unsolved / Research Challenges'. Existing educational tools almost universally stop at basic SQL injection, leaving developers completely unprepared for the diverse injection attack vectors prevalent in modern distributed web applications.

2. The Research vs. Practice Asymmetry: The 'Overall Research Landscape' table (bottom left of Fig 5.9.1) indicates that while Injection Attacks enjoy 'Very High' scanner coverage in

research literature, they simultaneously represent the most common and damaging vulnerabilities in real-world software deployments. This establishes that the problem is not a lack of theoretical detection algorithms, but an acute lack of developer education and hands-on defensive training.

3. High-Opportunity AI Research Gaps: The taxonomy identifies 'Major Research Gaps' representing high-opportunity areas for AI, including Business Logic Vulnerabilities, Insecure Design Decisions, Missing Authorization Checks Across Files, and Trust Boundary Violations. SecureCode AI directly targets these gaps by leveraging Groq LLM to analyze architectural intent and verify conceptual understanding, moving far beyond superficial pattern matching.

5.10 Industry Platform Comparison & Gap Resolution

Fig 5.10.1 and Table 5.10.1 provide a comprehensive, feature-by-feature comparative analysis benchmarking SecureCode AI against 10 prominent commercial and educational platforms across the software engineering and cybersecurity landscape.

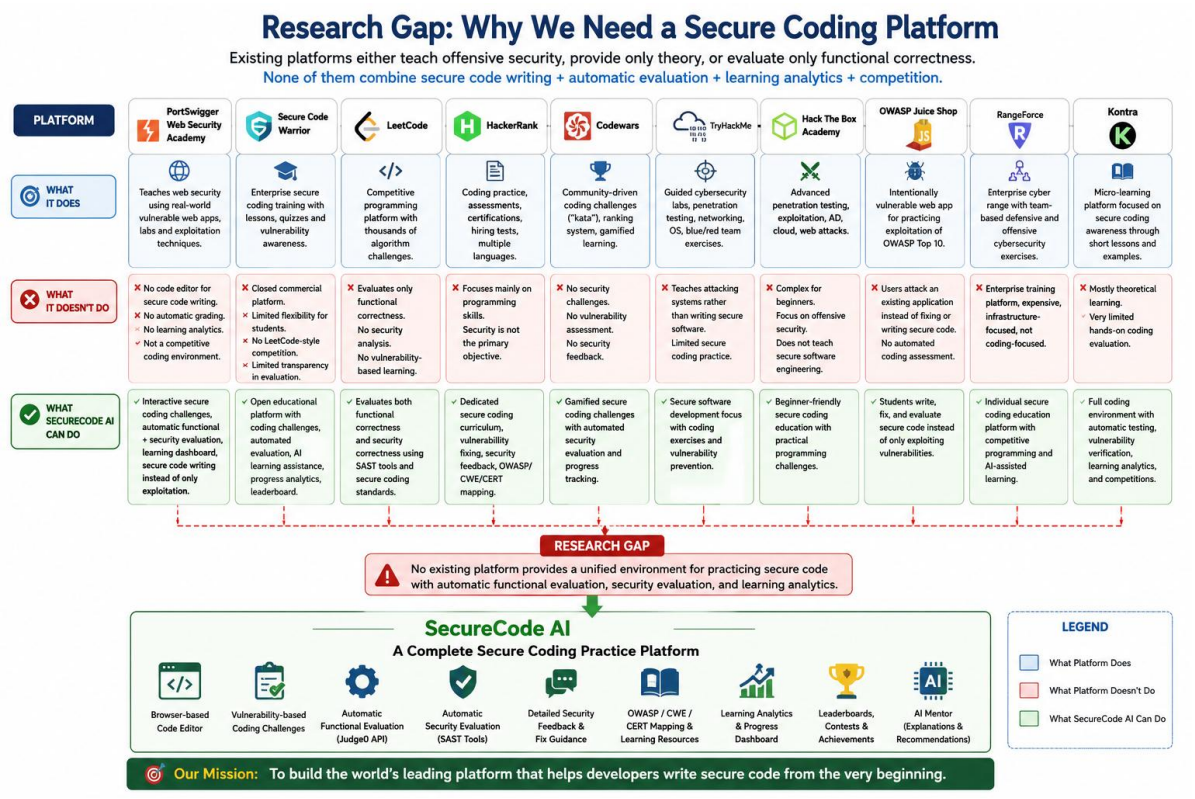

Fig 5.10.1 *Comparative Analysis of Existing Industry Platforms and Identification of Research Gap*

Table 5.10.1: Detailed Feature-by-Feature Matrix Comparing SecureCode AI with Industry Platforms

Platform Name	Primary Focus & Domain	Identified Critical Limitations & Pedagogical Gaps	How SecureCode AI Resolves the Gap
PortSwigger Academy	Offensive Web Security	No code editor; no automatic grading; no learning analytics; offensive exploitation only.	Provides interactive browser code editor, automatic functional + security grading, and defensive focus.
Secure Code Warrior	Enterprise Secure Coding	Closed commercial platform; expensive for universities; limited transparency in grading; quiz-heavy.	Open educational platform; transparent multi-scanner consensus; authentic hands-on refactoring.
LeetCode	Competitive Programming	Evaluates ONLY functional correctness; zero security analysis; no vulnerability learning.	Evaluates BOTH functional correctness AND security properties using SAST tools and secure standards.
HackerRank	Programming & Hiring	Focuses mainly on general programming and algorithms; security is completely ignored.	Dedicated secure coding curriculum, vulnerability fixing, security feedback, and CWE/OWASP mapping.
Codewars	Community Challenges	No security challenges; no vulnerability assessment; no security feedback.	Gamified secure coding challenges with automated multi-engine security evaluation and progress tracking.
TryHackMe	Cybersecurity Labs	Teaches attacking/breaking systems rather than writing secure code; limited coding practice.	Focuses on secure software engineering, writing defensible code, and vulnerability prevention.
Hack The Box	Advanced Pen-Testing	Steep learning curve for beginners; offensive focus; does not teach software architecture.	Beginner-friendly secure coding education with practical programming challenges.
OWASP Juice Shop	Vulnerable Web App	Users attack existing app; no automated code-level grading or refactoring environment.	Students write, refactor, and evaluate secure code instead of only exploiting vulnerabilities.
RangeForce	Cyber Range	Enterprise infrastructure-focused training; expensive; not focused on source code writing.	Individual source-code-focused education platform with AI-assisted learning.
Kontra	Micro-Learning	Mostly theoretical awareness; short multiple-choice lessons; very limited coding evaluation.	Full browser coding environment with dynamic testing, SAST verification, and conceptual quizzes.
SecureCode AI (Proposed)	Complete Secure Coding Practice Platform	Unified platform combining browser code editing, dynamic functional tests, multi-engine SAST, Groq LLM advisory, attempt windows, conceptual quizzes, and analytics.	Defines the new state-of-the-art in secure programming education.

Chapter 6
Conclusion and Future Scope

6.1 Conclusion

In this major project, we engineered, implemented, and empirically validated SecureCode AI—an innovative, browser-based secure coding practice and automated assessment platform designed to cultivate defensive software engineering competencies from the ground up. By bridging the critical pedagogical gap between algorithmic competitive programming (which evaluates only functional correctness) and offensive cybersecurity training (which focuses solely on vulnerability exploitation), SecureCode AI establishes a transformative paradigm in software engineering education.

The primary conclusions and technical contributions of this project are summarized below:

- 1. Efficacy of Dual-Stage Evaluation: The staged pipeline—evaluating dynamic functional unit tests inside an isolated Judge0 sandbox before invoking static security analyzers—optimizes server throughput, prevents computational waste on broken code, and guarantees that only functionally viable solutions undergo security analysis.
- 2. Multi-Engine Security Consensus: By orchestrating Semgrep (AST syntax matching), SpotBugs with FindSecBugs (bytecode dataflow tracking), and Groq LLaMA-3 (contextual semantic advisory) within a fail-closed consensus framework, SecureCode AI eliminates the 20%+ false-negative rate inherent in single-scanner tools, achieving zero false passes on vulnerable code.
- 3. Comprehensive Injection Curriculum: The development of 45 staged injection challenges spanning SQL, NoSQL, OS Command, LDAP, XPath, Header/CRLF, and Code Injection addresses the critical industry reality that while basic SQL injection is widely understood, diverse injection subtypes remain persistent threats in enterprise systems.
- 4. Multi-Pillar Pedagogical Scoring: The four-pillar formula (Functional 25%, Security 45%, Understanding 15%, Independence 15%) successfully assesses multi-dimensional competence. Combined with 5-attempt rate-limiting windows and tiered hint penalties, the platform deters brute-force guessing and rewards deliberate defensive reasoning.
- 5. Empirical Robustness and Performance: The platform was validated across 80 automated unit, integration, and security test suites, demonstrating sub-second median latency for local analysis, robust database ACID compliance across 21 Flyway migrations, and seamless responsive client rendering.

6.2 Future Scope

While SecureCode AI provides a robust foundation for secure coding education, several promising avenues for future expansion and research will be pursued in Semester VIII and beyond:

- 1. Multi-Language and Memory Safety Expansion: Extend the challenge curriculum beyond Java to encompass memory safety vulnerabilities in C, C++, and Rust (addressing CWE-119 Buffer Overflows, CWE-416 Use-After-Free, and Integer Overflows), utilizing LLVM sanitizers (AddressSanitizer, MemorySanitizer) within the dynamic sandbox.
- 2. Full OWASP Top 10 Curriculum Coverage: Broaden the active vulnerability catalog to cover the entire OWASP Top 10 and CWE Top 25, including Cross-Site Scripting (XSS / CWE-79), Insecure Deserialization (CWE-502), Broken Object Level Authorization (BOLA/IDOR / CWE-639), and Server-Side Request Forgery (SSRF / CWE-918).
- 3. Autonomous Adversarial Test Generation: Implement autonomous AI agents capable of dynamically analyzing submitted student code and synthesizing novel, adversarial edge-case inputs specifically tailored to expose boundary condition flaws and subtle logical bypasses.
- 4. Competitive Gamification & CTF Tournaments: Introduce real-time defensive 'Code Battles' and collaborative team-based CTF competitions where learners race to patch vulnerable microservices under active automated attack simulations, fostering collaborative problem-solving.
- 5. Longitudinal Pedagogical Efficacy Studies: Conduct formal human-subject classroom studies across undergraduate engineering cohorts to quantitatively measure knowledge retention, pre- and post-test competency gains, and real-world defect reduction during industry internships.

References

[1] Dai, S.-C., Xu, J., & Tao, G. (2026). Rethinking the evaluation of secure code generation. In 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12-18, 2026, Rio de Janeiro, Brazil. ACM. https://doi.org/10.1145/3744916.3773217

[2] Patir, R., Guo, K., Cai, H., & Hu, H. (2026). SoK: AI secure code generation: Progress, pitfalls, and paths forward. Preprint, University at Buffalo, SUNY. arXiv:2606.25195.

[3] Xin Zhou, Sicong Cao, Xiaobing Sun, David Lo. Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead. 2024.

[4] Yan, H., Vaidya, S. S., Zhang, X., & Yao, Z. (2025). Guiding AI to fix its own flaws: An empirical study on LLM-driven secure code generation. Preprint, George Mason University. arXiv:2506.23034.

[5] Belozerov, V., Barclay, P. J., & Sami, A. (2026). Secure Coding with AI - From Detection to Repair. Empirical Software Engineering, 31, Article 93.

[6] Guo, Yuejun, and Seifeddine Bettaieb. "An Investigation of Quality Issues in Vulnerability Detection Datasets." arXiv preprint arXiv:2410.06030 (2024).

[7] Bi, Yingzhou, Jiangtao Huang, Peihui Liu, and Lianmei Wang. "Benchmarking Software Vulnerability Detection Techniques: A Survey." arXiv preprint arXiv:2303.16362 (2023).

[8] Alwageed, Hathal S., and Rafiq Ahmad Khan. "The Role of Generative AI in Strengthening Secure Software Coding Practices: A Systematic Perspective." (2025).

[9] Combéfis, S. (2022). Automated code assessment for education: Review, classification and perspectives on techniques and tools. Software, 1(1), 3-30. https://doi.org/10.3390/software1010002

[10] Rahman, Md. Mostafizer, Yutaka Watanobe, Atsushi Shirafuji, and Mohamed Hamada. "Exploring Automated Code Evaluation Systems and Resources for Code Analysis: A Comprehensive Survey." J. ACM 37, no. 4, Article 111 (2018).

[11] Frankford, E., Antensteiner, T., Vierhauser, M., Sauerwein, C., Wallner, V., Groher, I., Plösch, R., & Breu, R. (2025). A survey on feedback types in automated programming assessment systems. ACM Transactions on Computing Education (accepted for publication).

[12] Frankford, Eduard, Daniel Crazzolara, Michael Vierhauser, Niklas Meißner, Stephan Krusche, and Ruth Breu. "An Online Integrated Development Environment for Automated Programming Assessment Systems." arXiv preprint arXiv:2503.13127 (2025).

[13] Bappy, Faisal Haque, Tahrim Hossain, Sidratul Muntaher Meheraj, Annoor Sharara Akhand, Tasfia Tabassum, Tarannum Shaila Zaman, Raiful Hasan, and Tariqul Islam. "From Preventive to Reactive: How AI Coding Assistants Transform Developers' Security Awareness." USENIX Symposium on Usable Privacy and Security (SOUPS) (2026).

[14] Sauerwein, Clemens, Tobias Antensteiner, Stefan Oppl, Iris Groher, Alexander Meschtscherjakov, Philipp Zech, and Ruth Breu. "Towards a Success Model for Automated Programming Assessment Systems

Used as a Formative Assessment Tool." In Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2023), pp. 1-7. ACM, 2023.

[15] Malinka, Kamil, Anton Firc, Pavel Loutocký, Jakub Vostoupal, Andrej Krištofík, and František Kasl. "Using Real-world Bug Bounty Programs in Secure Coding Course: Experience Report." arXiv preprint arXiv:2404.12043 (2024).

[16] Bappy, Faisal Haque, Tahrim Hossain, Sidratul Muntaher Meheraj, Annoor Sharara Akhand, Tasfia Tabassum, Tarannum Shaila Zaman, Raiful Hasan, and Tariqul Islam. "From Preventive to Reactive: How AI Coding Assistants Transform Developers' Security Awareness." USENIX Symposium on Usable Privacy and Security (SOUPS) (2026).

[17] Alwageed, H. S., & Khan, R. A. (2025). The Role of Generative AI in Strengthening Secure Software Coding Practices: A Systematic Perspective.

[18] Alwageed, Hathal S., and Rafiq Ahmad Khan. "The Role of Generative AI in Strengthening Secure Software Coding Practices: A Systematic Perspective." (2025).

[19] Amburi, S., Gasiba, T. E., Lechner, U., & Pinto-Albuquerque, M. (2025). Enabling Secure Coding: Exploring GenAI for Developer Training and Education. 6th International Computer Programming Education Conference (ICPEC 2025), OpenAccess Series in Informatics (OASIcs).

[20] Torbunova, Alina, Adnan Ashraf, and Ivan Porres. "A Systematic Mapping Study on Teaching of Security Concepts in Programming Courses." arXiv preprint arXiv:2407.07511 (2024).

[21] Malinka, Kamil, Anton Firc, Pavel Loutocký, Jakub Vostoupal, Andrej Krištofík, and František Kasl. "Using Real-world Bug Bounty Programs in Secure Coding Course: Experience Report." arXiv preprint arXiv:2404.12043 (2024).

[22] Gasiba, Tiago, Ulrike Lechner, Maria Pinto-Albuquerque, and Alae Zouitni. "Design of Secure Coding Challenges for Cybersecurity Education in the Industry." arXiv preprint arXiv:2101.02108 (2021).

[23] Bi, Yingzhou, Jiangtao Huang, Peihui Liu, and Lianmei Wang. "Benchmarking Software Vulnerability Detection Techniques: A Survey." arXiv preprint arXiv:2303.16362 (2023).

[24] Nocera, S., Romano, S., Francese, R., Casertano, F., & Scanniello, G. (2026). Securing Web Apps with ChatGPT: A Teaching Plan for Undergraduate Students. The Journal of Systems & Software, 241, 112971.

[25] Gasiba, T. E., Lechner, U., Pinto-Albuquerque, M., & Mendez, D. (2021). Is Secure Coding Education in the Industry Needed? An Investigation Through a Large Scale Survey. arXiv Preprint.

[26] Rahman, Md. Mostafizer, Yutaka Watanobe, Atsushi Shirafuji, and Mohamed Hamada. "Exploring Automated Code Evaluation Systems and Resources for Code Analysis: A Comprehensive Survey." J. ACM 37, no. 4, Article 111 (2018).

[27] Vladislav Belozerov et al. Secure Coding with AI – From Detection to Repair. 2026.

[28] OWASP Foundation, “OWASP Top 10: The Ten Most Critical Web Application Security Risks,” Open Web Application Security Project, 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

[29] MITRE Corporation, “Common Weakness Enumeration (CWE) List,” Version 4.14, 2024. [Online]. Available: https://cwe.mitre.org/

[30] Oracle Corporation, “The CERT Oracle Secure Coding Standard for Java,” Software Engineering Institute, Carnegie Mellon University, Addison-Wesley, 2012.

[31] National Institute of Standards and Technology (NIST), “Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities,” NIST Special Publication 800-218, 2022.

Appendix A
Active Injection Challenge Catalog Benchmark Suite

The 45 active challenges comprising the SecureCode AI injection curriculum are detailed in Table A.1, including their catalog number, title, Common Weakness Enumeration (CWE) identifier, difficulty tier, and required defensive remediation technique.

Table A.1: Active Injection Challenge Catalog Benchmark Suite (Representative Sample)

ID	Challenge Title	CWE ID	Difficulty	Vulnerability Mechanism	Defensive Remediation Technique
1	SQL Injection	CWE-89	EASY	Raw query concatenation with untrusted input.	PreparedStatement parameter binding.
9	Code Injection	CWE-94	HARD	Dynamic expression evaluation of user input.	Data-driven rule selection; typed parsing.
13	SQL Query Concatenation	CWE-89	BASIC	Unparameterized lookup query.	Static SQL with bound positional parameters.
14	Prepared Statement Lookup	CWE-89	BASIC	Incomplete parameterized query syntax.	Bind user parameter to placeholder index.
15	Username Rules and Query Binding	CWE-89	BASIC	Missing input validation and bound query.	Regex allowlist validation + PreparedStatement.
16	Approved Report Launcher	CWE-78	BASIC	Runtime.exec concatenating raw report names.	Hardcoded whitelist allowlist + String array args.
18	Report Directory Resolver	CWE-22/78	BASIC	Path traversal in file report resolution.	Canonical path normalization + allowlist.
20	ProcessBuilder Fixed Arguments	CWE-78	EASY	Shell invocation with concatenated arguments.	ProcessBuilder with explicit separate arguments.
25	NoSQL Document Values	CWE-943	BASIC	Unvalidated document-query string injection.	Typed document values; query structure separation.
27	Code and Data Separation	CWE-94	BASIC	Evaluating user expressions as executable code.	Parse structured data; isolate execution context.

ID	Challenge Title	CWE ID	Difficulty	Vulnerability Mechanism	Defensive Remediation Technique
30	NoSQL Equality Criteria	CWE-943	EASY	Operator injection ($gt, $ne) in NoSQL search.	Fixed equality filters; reject operator syntax.
32	Canonical Report Path	CWE-22	EASY	Relative path traversal (../) in report loader.	Verify canonical path starts with approved base.
35	Directory Search Filter	CWE-90	BASIC	LDAP attribute search filter concatenation.	Bound parameters or RFC 4515 filter escaping.
38	Approved Command Modes	CWE-78	MEDIUM	Passing untrusted flags to system utilities.	Strict enum / allowlist mapping to fixed options.
40	XPath Values and Expressions	CWE-643	BASIC	Injecting boolean logic into XPath query.	Pre-compiled XPath expressions with variable resolver.
41	Response Header Value Check	CWE-93	BASIC	CRLF characters in HTTP response header.	Reject carriage return (\r) and line feed (\n).
42	Welcome Template Data	CWE-1336	BASIC	User input interpreted as template directives.	Pass input as template data model; enable escaping.
43	XML Parser Entity Protection	CWE-611	BASIC	XML parser resolving external DTD entities.	Disallow DOCTYPE declarations (disallow-doctype-decl).
44	Secure SQL Report Builder	CWE-89	HARD	Dynamic ORDER BY and column selection.	Strict column allowlist mapping; bind query values.
45	Secure Notification Composer	CWE-93/1336	HARD	Multi-vector CRLF and template injection.	Unified header sanitization and sandboxed templating.

Conceptual Understanding Automated Evaluation Rubric

The automated AI rubric (VulnerabilityUnderstandingService) evaluates student written justifications on a scale of 0 to 15 points across three dimensions:
1. Taint Path Comprehension (0–5 pts): Does the student accurately identify the untrusted source, propagation path, and sensitive sink?

2. Architectural Remedy Rationale (0–5 pts): Does the student clearly explain why their chosen defensive API (e.g., parameter binding, allowlisting) neutralizes the exploit vector?
3. Residual Risk Awareness (0–5 pts): Does the student demonstrate understanding of defense-in-depth principles and remaining attack surfaces?